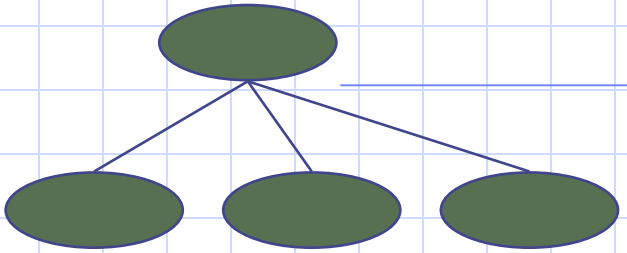
Graphs

sequence/linear (1 to 1)

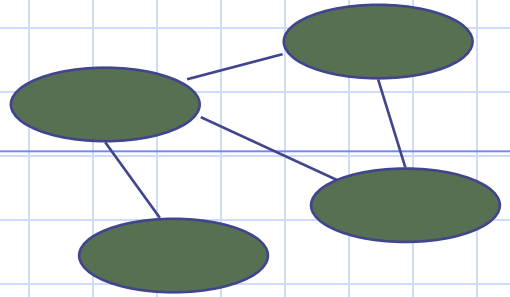


first ith last

hierarchical
(1 to many)

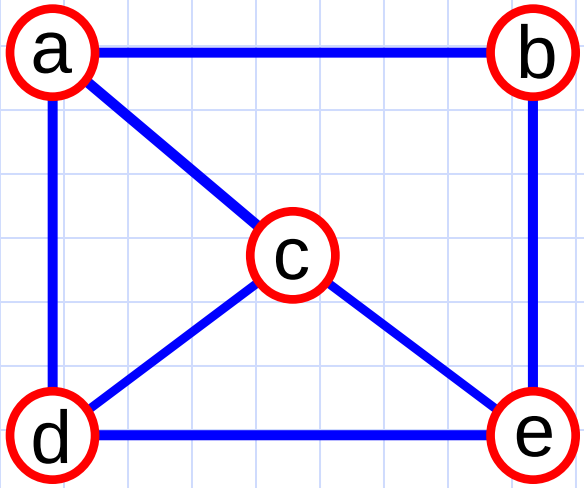


graph (many to many)



What is a Graph?

- A graph is a pair (V, E), where
  - V is a set of nodes, called vertices
  - E is a collection of pairs of vertices, called edges
- V(G) and E(G) represent the sets of vertices and edges of G, respectively
- Example:



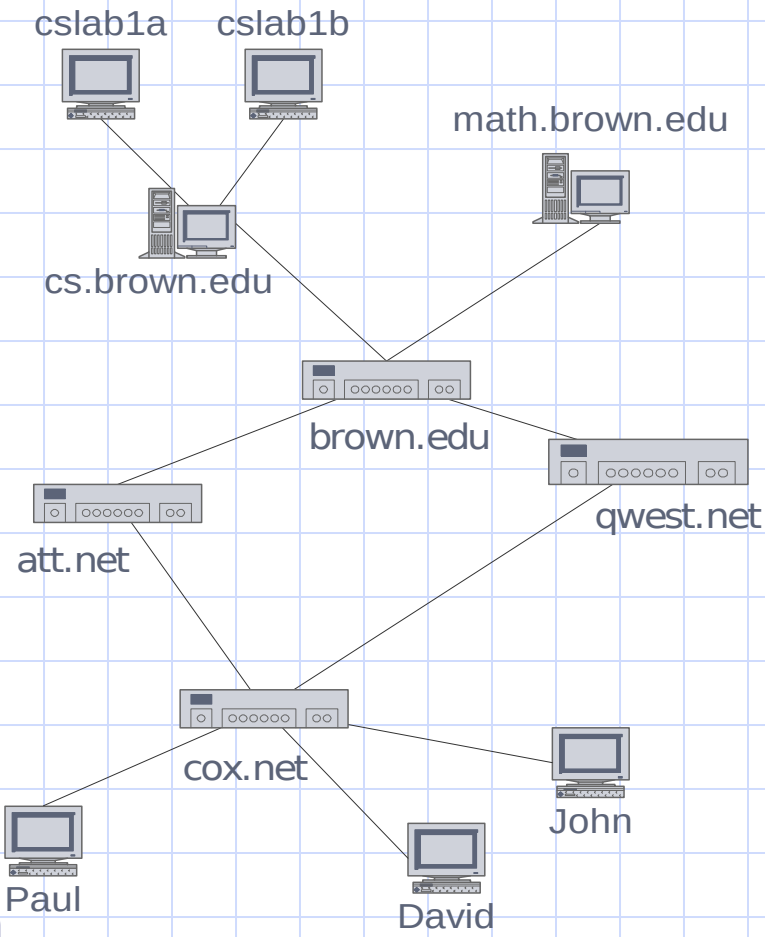
V= {a, b, c, d, e}

E= {(a,b), (a,c), (a,d), (b,e), (c,d), (c,e), (d,e)}

- A tree is a special type of graph!
- A tree is a graph that is connected and acyclic.

Applications

- Electronic circuits
  - Printed circuit board
  - Integrated circuit
- Transportation networks
  - Highway network
  - Flight network
- Computer networks
  - Local area network
  - Internet
  - Web
- Databases
  - Entity-relationship diagram

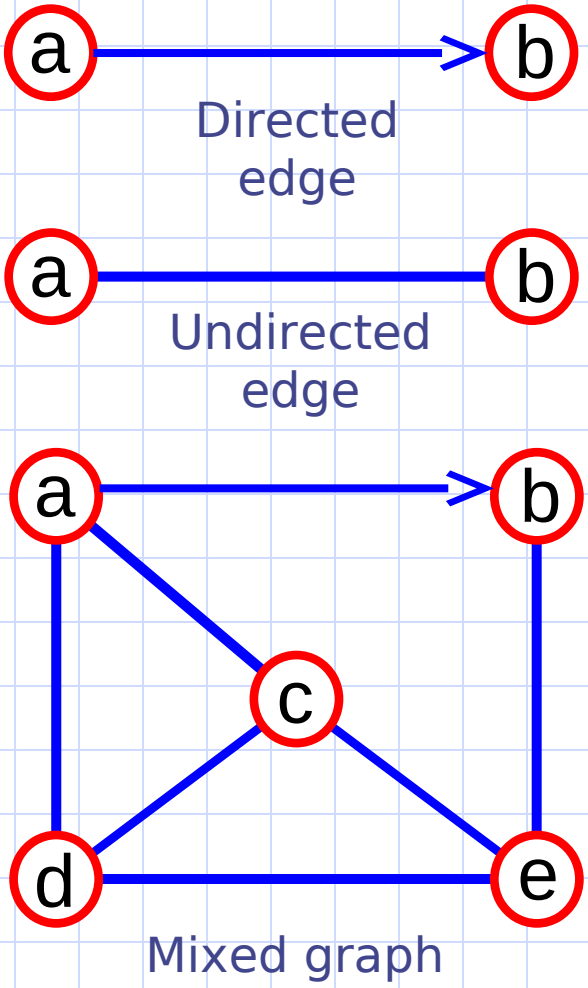


What can we do with graphs?

- Find a *path* from one place to another
- Find the *shortest path* from one place to another
- Determine connectivity
- Find the “weakest link” (min cut)
  - check amount of redundancy in case of failures
- Find the amount of flow that will go through them

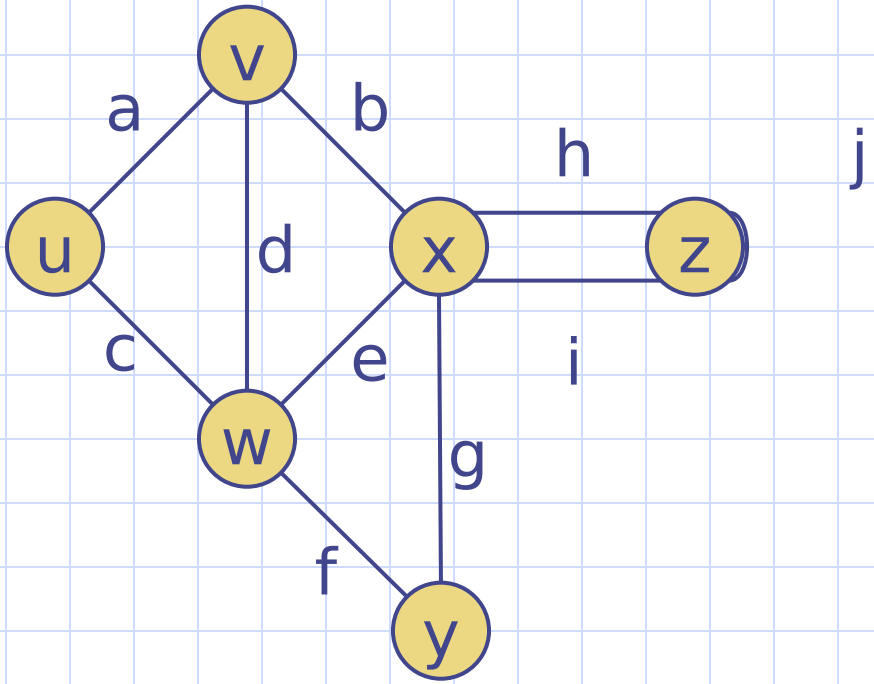
Edge and Graph Types

- Directed edge
  - ordered pair of vertices (u, v)
  - first vertex u is the origin
  - second vertex v is the destination
- Undirected edge
  - unordered pair of vertices (u, v)
- Directed graph (Digraph)
  - all the edges are directed
  - e.g., route network
- Undirected graph
  - all the edges are undirected
  - e.g., flight network
- Mixed graph
  - some edges are undirected and some edges are directed
  - e.g., a graph modeling a city map



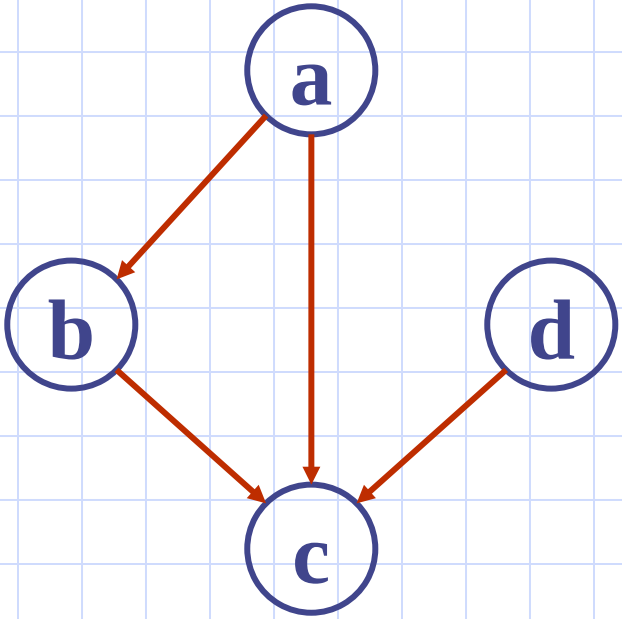
Terminology

- End vertices (or endpoints) of an edge
  - u and v are the *endpoints* of a
- Edges incident to a vertex
  - a, d, and b are *incident* to v
- Adjacent vertices
  - u and v are *adjacent*
- Degree of a vertex
  - x has *degree* 5
- Parallel edges
  - h and i are *parallel edges*
- Self-loop
  - j is a *self-loop*



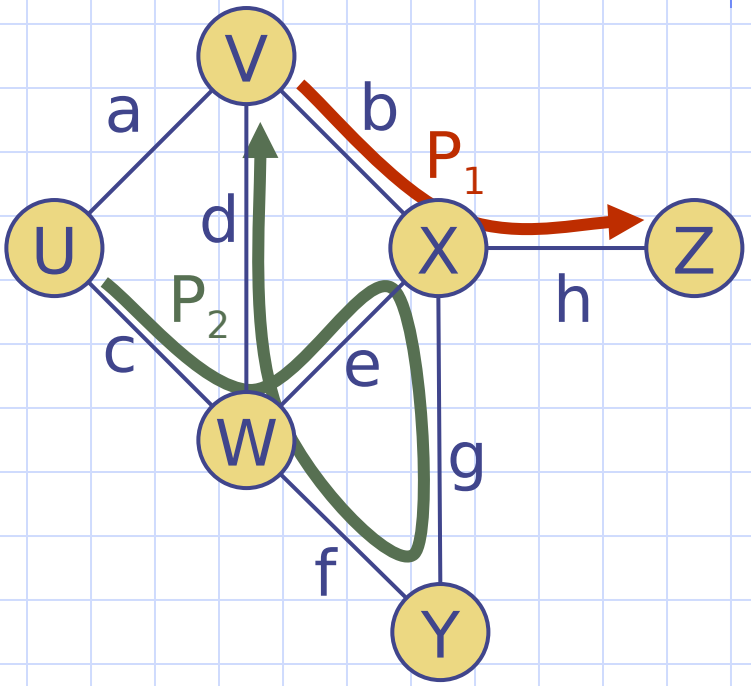
Terminology (cont.)

- Outgoing edges of a vertex
  - (a, b) and (a, c) are outgoing edges of vertex a
- Incoming edges of a vertex
  - (b, c), (d, c) and (a, c) are incoming edges of vertex c
- In-degree of a vertex
  - c has *in-degree* 3
  - b has *in-degree* 1
- Out-degree of a vertex
  - a has *out-degree* 2
  - b has *out-degree* 1



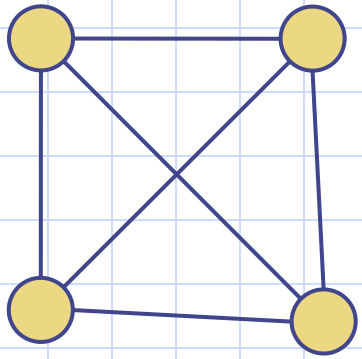
Terminology (cont.)

- Path
  - sequence of alternating vertices and edges
  - begins with a vertex
  - ends with a vertex
  - each edge is preceded and followed by its endpoints
- Simple path
  - path such that all its vertices and edges are distinct
- Examples
  - P_1=(V, b, X, h, Z) is a simple path
  - P_2=(U, c, W, e, X, g, Y, f, W, d, V) is a path that is not simple



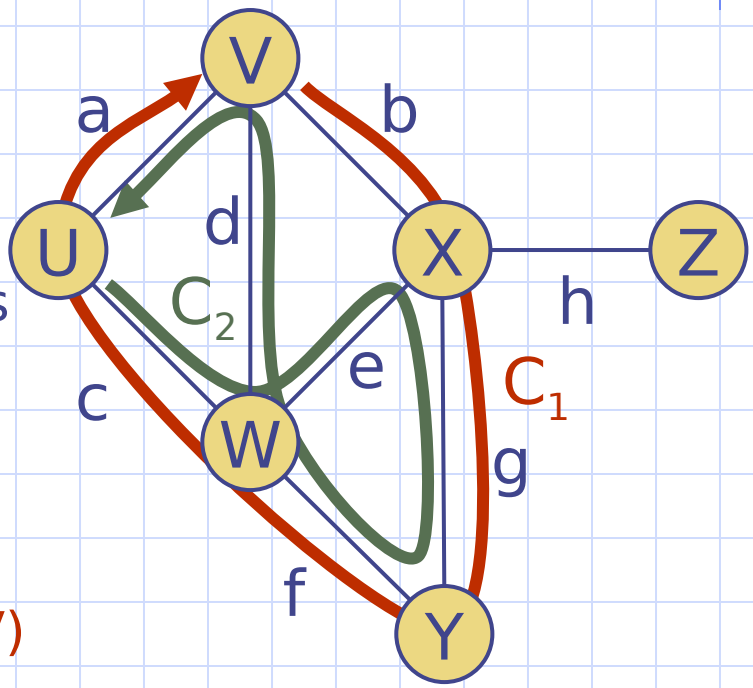
Terminology (cont.)

- *Dense* graph: $|E| \approx |V|^2$; *Sparse* graph: $|E| \approx |V|$
- A *weighted graph* associates weights with either the edges or the vertices
- A complete graph is a graph that has the maximum number of edges
  - for undirected graph with n vertices, the maximum number of edges is n(n-1)/2
  - for directed graph with n vertices, the maximum number of edges is n(n-1)



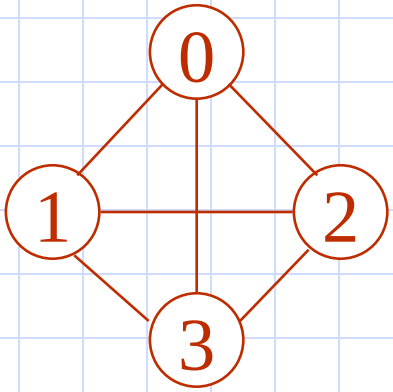
Terminology (cont.)

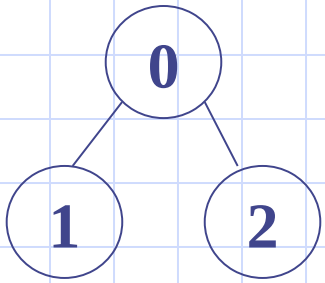
- Cycle
  - A cycle is a path whose start and end vertices are the same
  - each edge is preceded and followed by its endpoints
- Simple cycle
  - A cycle is simple if each edge is distinct and each vertex is distinct, except for the first and the last one
- Examples
  - C_1=(V, b, X, g, Y, f, W, c, U, a, V) is a simple cycle
  - C_2=(U,c,W,e,X,g,Y,f,W,d,V,a,U) is a cycle that is not simple



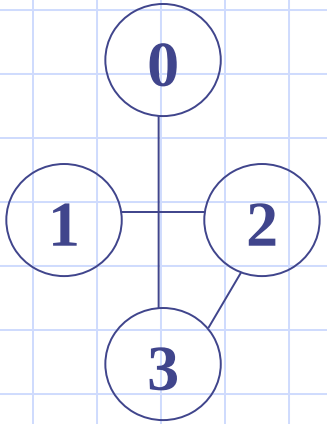
Terminology (cont.)

- A subgraph of G is a graph G' such that
  - V(G') is a subset of V(G) [V(G') ⊆ V(G)] and
  - E(G') is a subset of E(G) [E(G') ⊆ E(G)]
- A spanning subgraph G' of G is a subgraph of G that contains all the vertices of G, that is
  - V(G') is equal to V(G) [V(G') = V(G)] and
  - E(G') is a subset of E(G) [E(G') ⊆ E(G)]
- A forest is a graph without cycles.
- A (free) tree is a connected forest, that is, a connected graph without cycles.
- A spanning tree of a graph G is a spanning subgraph that is a (free) tree.





A subgraph



A spanning subgraph (tree)

Properties

Property 1

For an undirected graph

$\sum_v \deg(v) = 2m$

Proof: each edge is counted twice

Property 2

For a directed graph

$\sum_v \text{indeg}(v) = \sum_v \text{outdeg}(v) = m$

Proof: each edge is counted once for in-degree and once for out-degree

Property 3

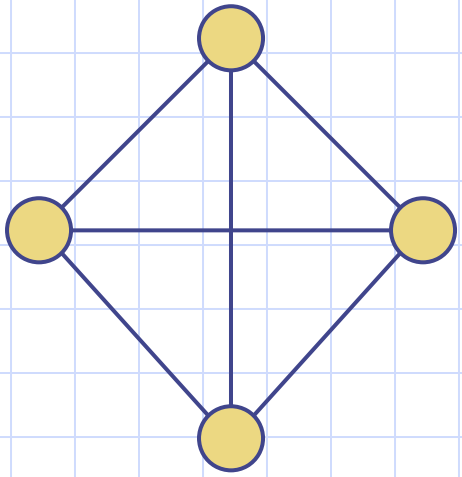
If G is a simple undirected graph, then $m \leq n(n-1)/2$, and

if G is a simple directed graph, then $m \leq n(n-1)$.

Proof: each vertex has degree at most $(n-1)$. Then use Property 1 and Property 2.

Notation

n	number of vertices
m	number of edges
$\deg(v)$	degree of vertex v

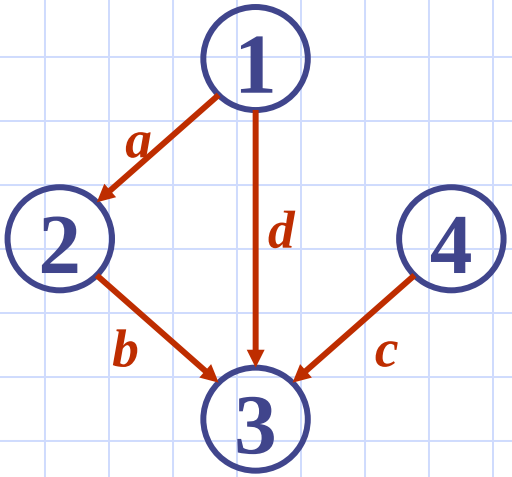


Graph Representation

- For graphs to be computationally useful, they have to be conveniently represented in programs
- There are two computer representations of graphs:
  - **Adjacency matrix representation**
  - **Adjacency lists representation**

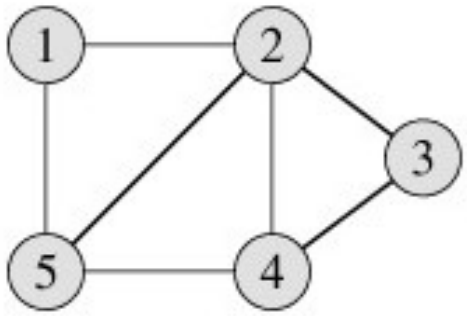
Adjacency Matrix Representation

- Assume V = {1, 2, …, n}
- An *adjacency matrix* represents the graph as a n x n matrix A:
  - A[i, j] = 1 if edge (i, j) ∈ E (or weight of edge)
    = 0 if edge (i, j) ∉ E



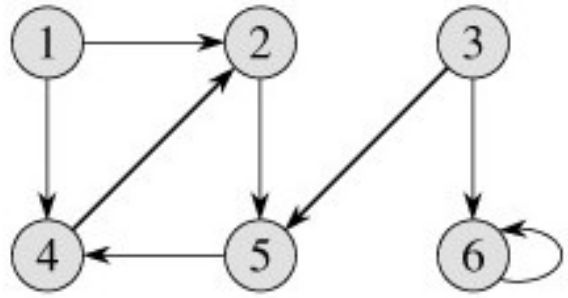
A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

Adjacency Matrix Representation



Undirected Graph

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0



Directed Graph

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

The adjacency matrix for an undirected graph is symmetric; the adjacency matrix for a digraph need not be symmetric

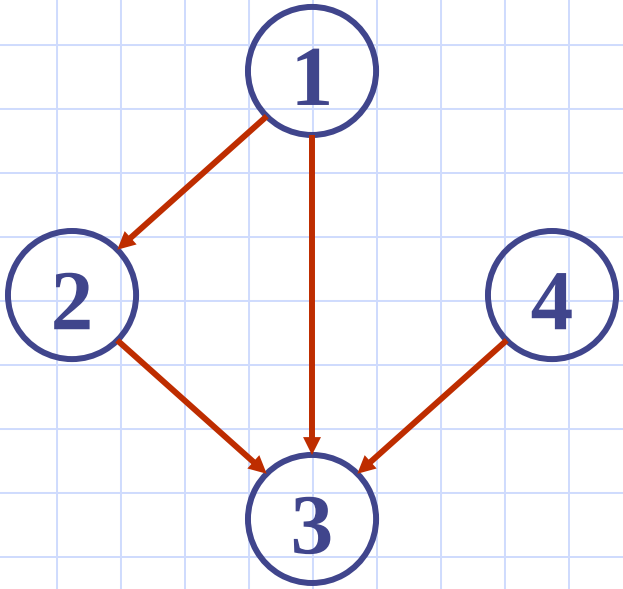
Adjacency Matrix Representation

- Pros:
  - Simple to implement
  - Easy and fast to tell if a pair (i, j) is an edge: simply check if A[i, j] is 1 or 0
  - Can be very efficient for small graphs

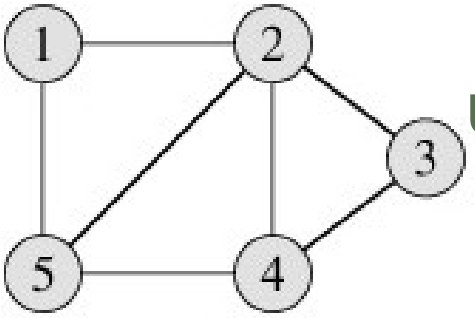
- Cons:
  - No matter how few edges the graph has, the matrix takes $O(n^2)$ in memory

Adjacency Lists Representation

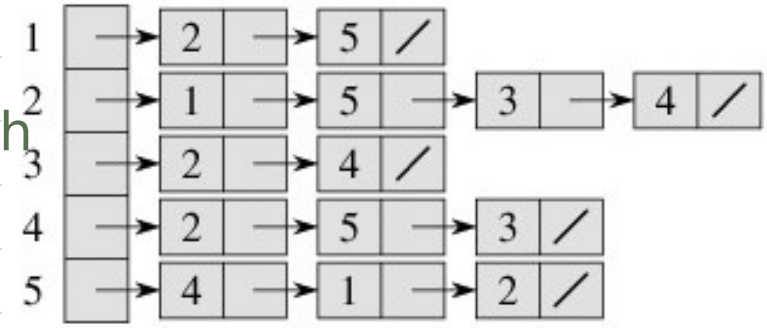
- A graph is represented by a one-dimensional array L of linked lists, where
  - L[i] is the linked list containing all the nodes adjacent to node i.
  - The nodes in the list L[i] are in no particular order

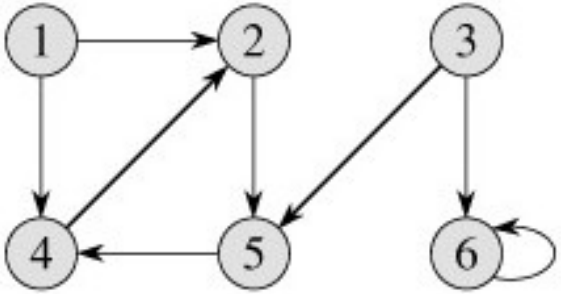


Adjacency Lists Representation

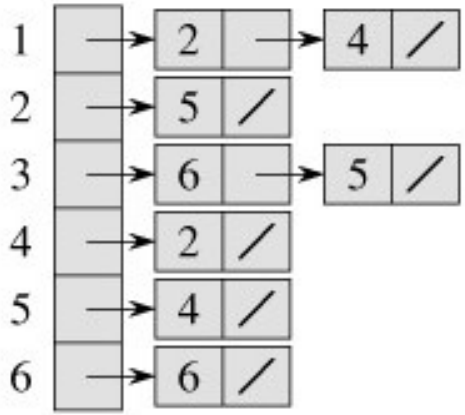


Undirected Graph





Directed Graph

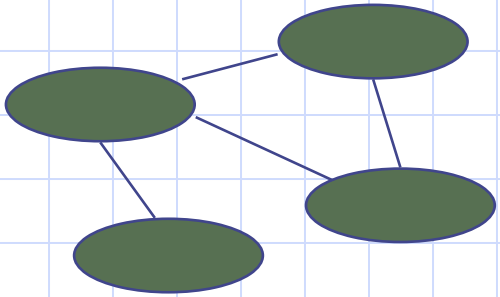


Adjacency Lists Representation

- Pros:
  - Saves on space (memory): the representation takes O(|V|+|E|) memory.
  - Good for large, sparse graphs (e.g., planar maps)
- Cons:
  - It can take up to O(n) time to determine if a pair of nodes (i, j) is an edge: one would have to search the linked list L[i], which takes time proportional to the length of L[i].

Graphs (Cont.)

graph (many to many)



Graph Searching

- Given: a graph G = (V, E), directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a tree on the graph
  - Pick a vertex as the root
  - Choose certain edges to produce a tree
  - Note: might also build a *forest* if graph is not connected

- There are two standard graph traversal techniques:
  - Breadth-First Search (BFS)
  - Depth-First Search (DFS)

Breadth-First Search

- "Explore" a graph, turning it into a tree
  - One vertex at a time
  - Expand frontier of explored vertices across the *breadth* of the frontier

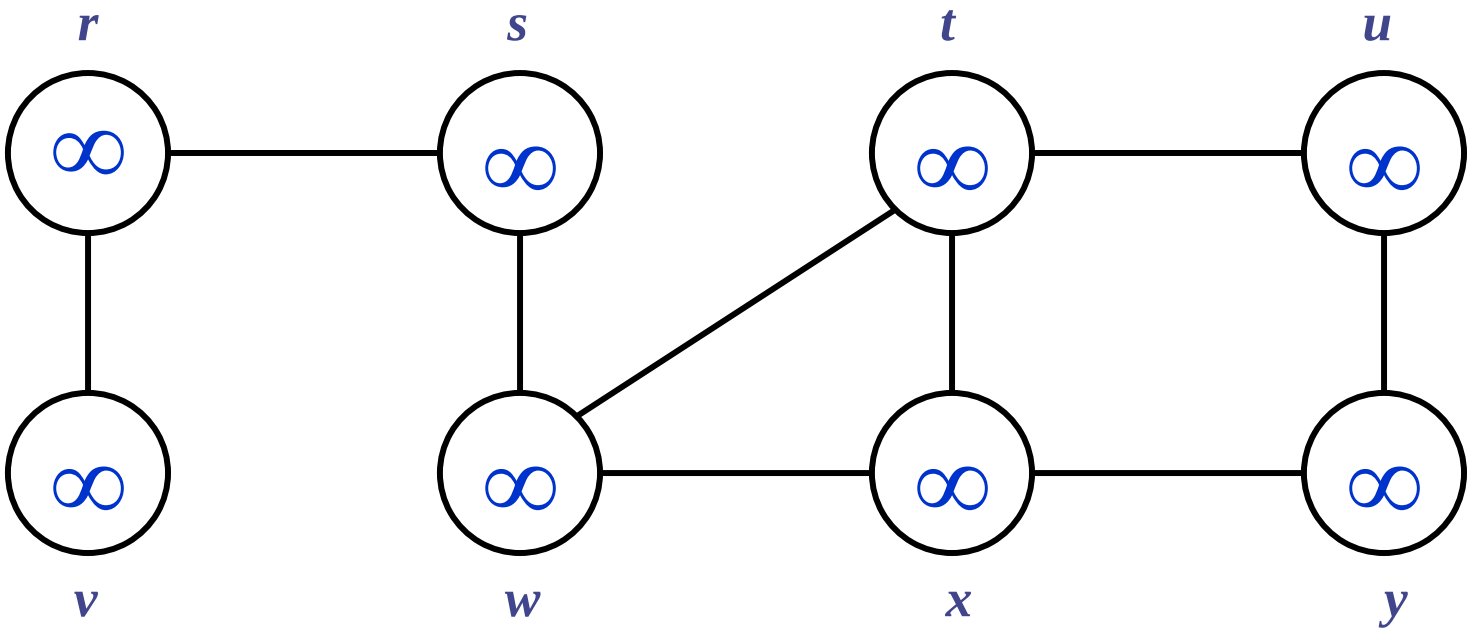
- Builds a tree over the graph
  - Pick a *source vertex* to be the root
  - Find ("discover") its children, then their children, etc.

Breadth-First Search

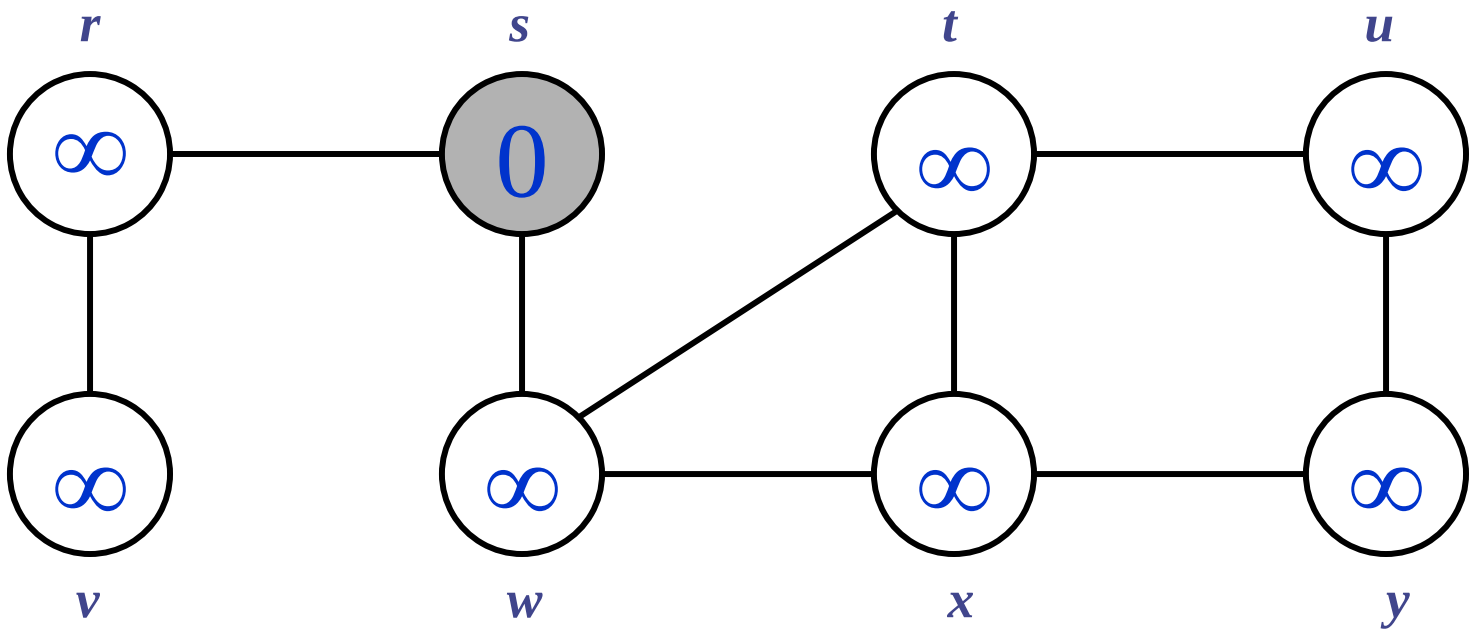
- Again will associate vertex “colors” to guide the algorithm
  - White vertices have not been discovered
    - All vertices start out white
  - Grey vertices are discovered but not fully explored
    - They may be adjacent to white vertices
  - Black vertices are discovered and fully explored
    - They are adjacent only to black and grey vertices
- Explore vertices by scanning adjacency list of grey vertices

```
BFS(G, s)
 1  for each vertex u ∈ V[G] − {s}
 2       do color[u] ← WHITE
 3          d[u] ← ∞
 4          π[u] ← NIL
 5  color[s] ← GRAY
 6  d[s] ← 0
 7  π[s] ← NIL
 8  Q ← ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11       do u ← DEQUEUE(Q)
12          for each v ∈ Adj[u]
13               do if color[v] = WHITE
14                    then color[v] ← GRAY
15                         d[v] ← d[u] + 1
16                         π[v] ← u
17                         ENQUEUE(Q, v)
18          color[u] ← BLACK
```

Breadth-First Search: Example

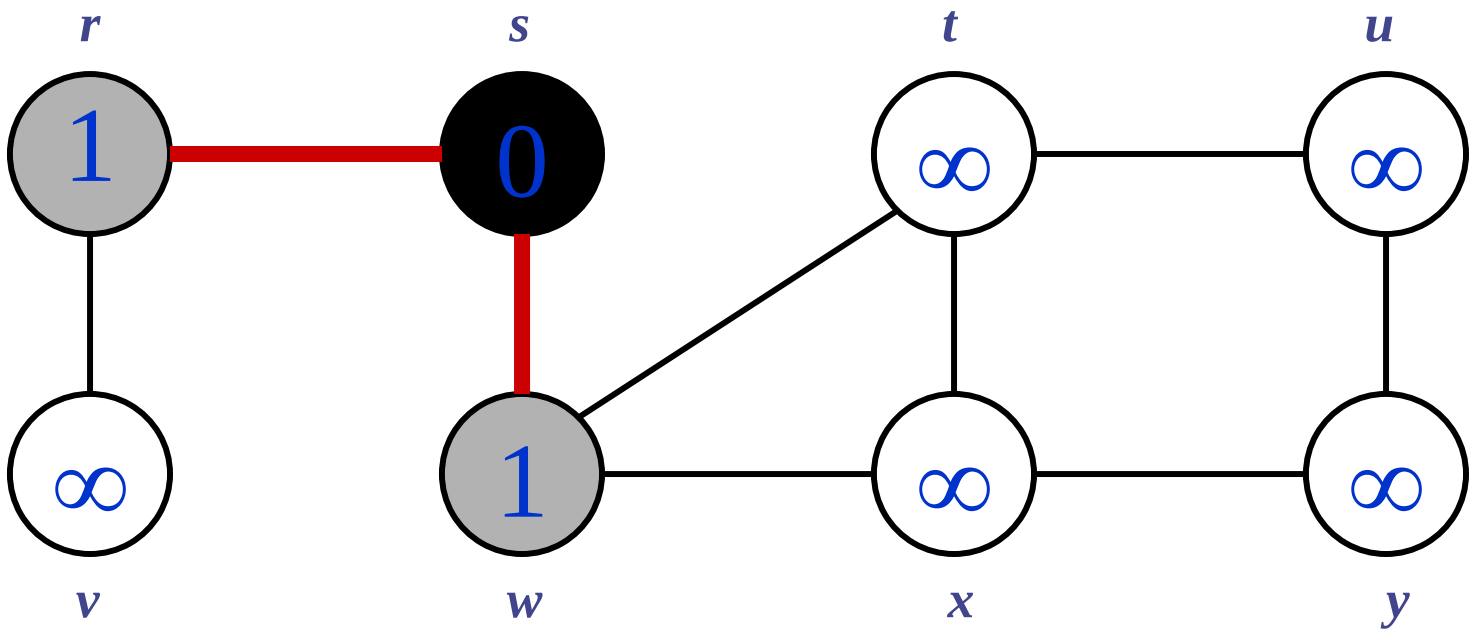


Breadth-First Search: Example



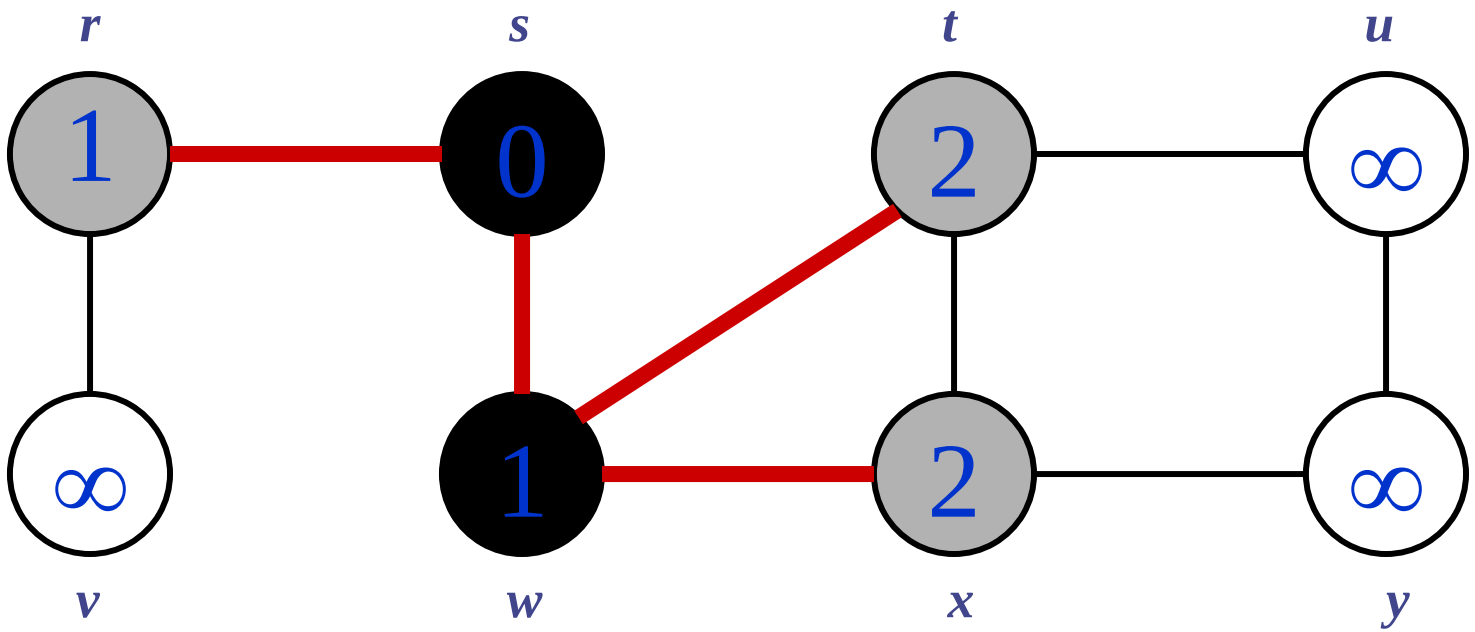
$Q:$ | s |

Breadth-First Search: Example



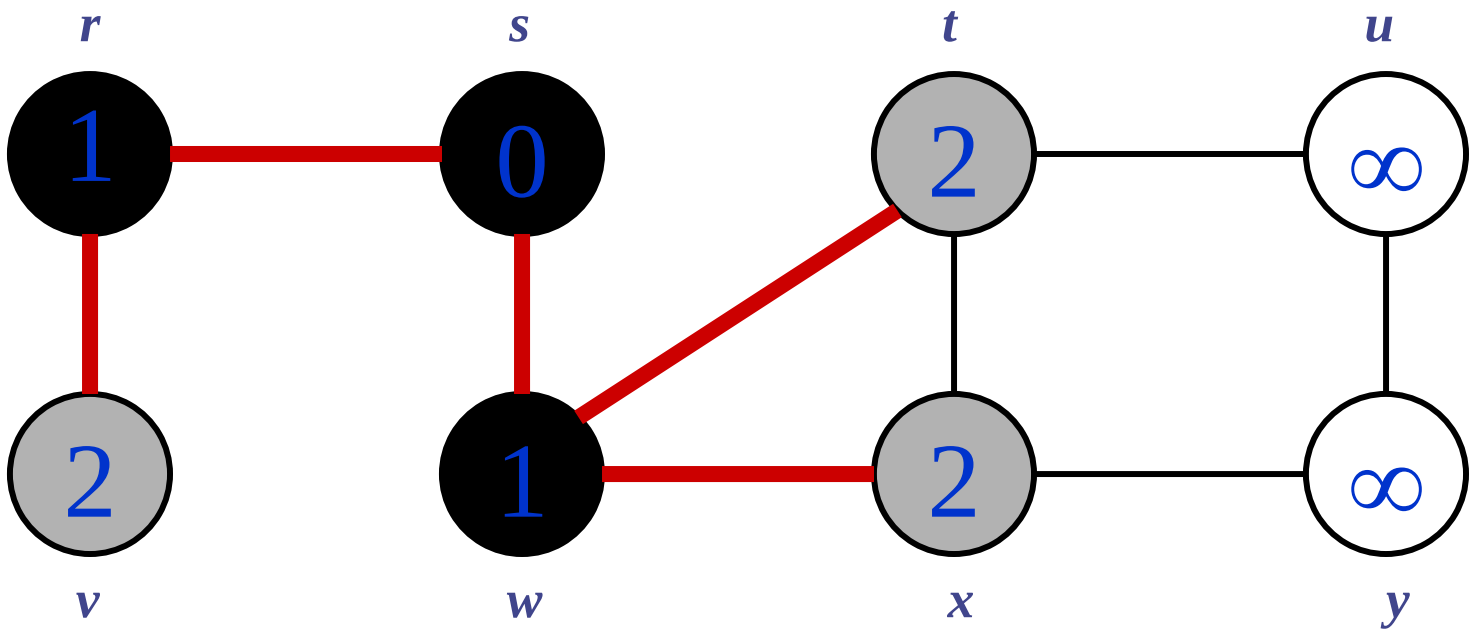
Q: | w | r |

Breadth-First Search: Example



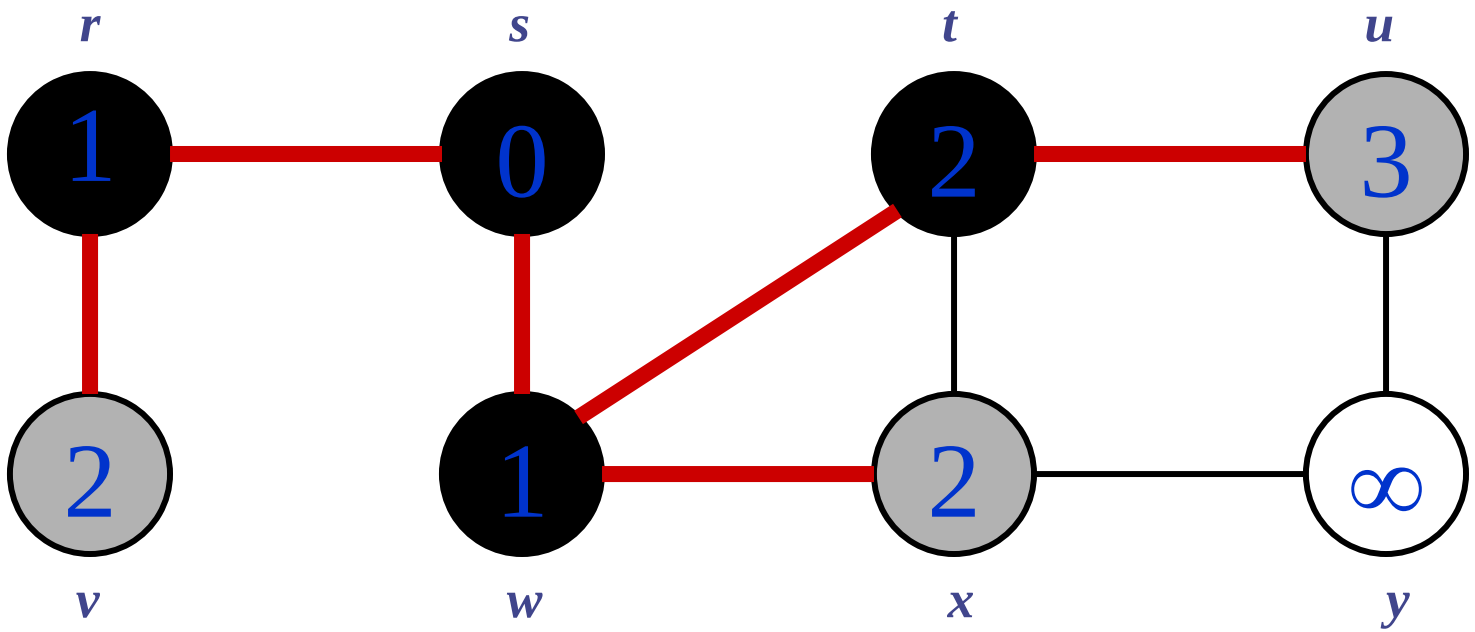
Q: | r | t | x |

Breadth-First Search: Example



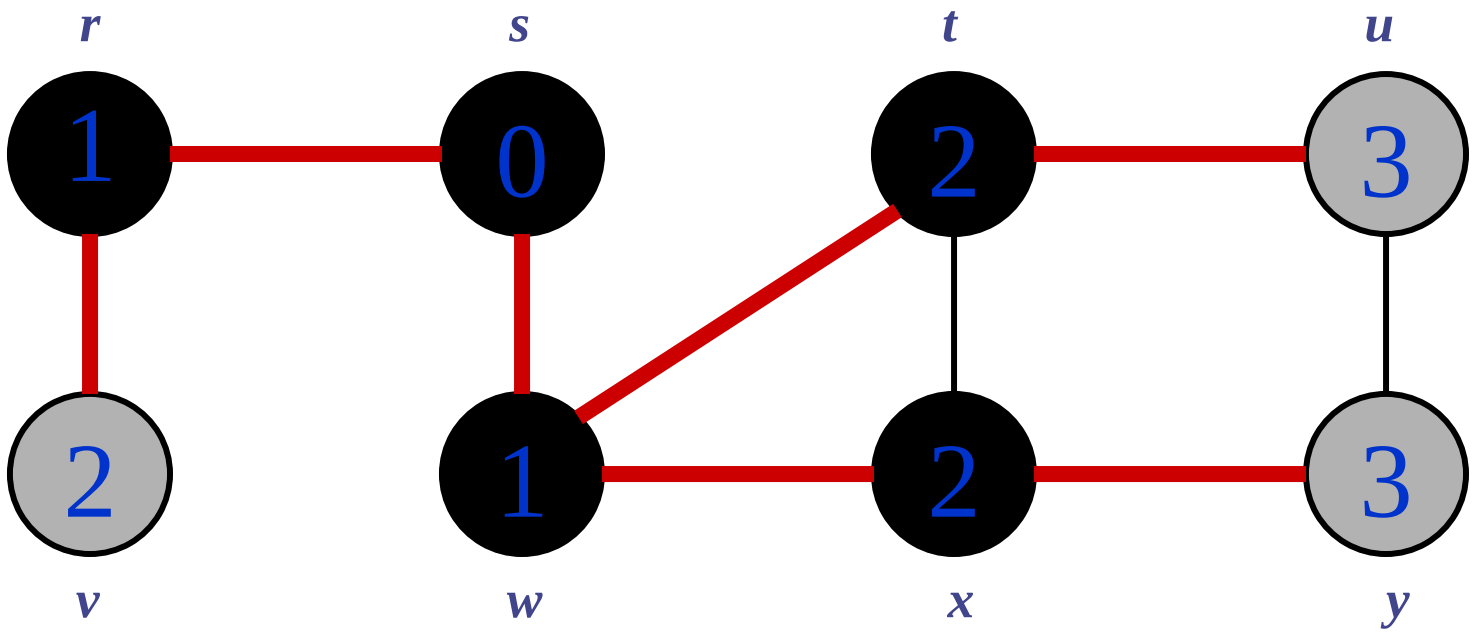
Q: | t | x | v |

Breadth-First Search: Example



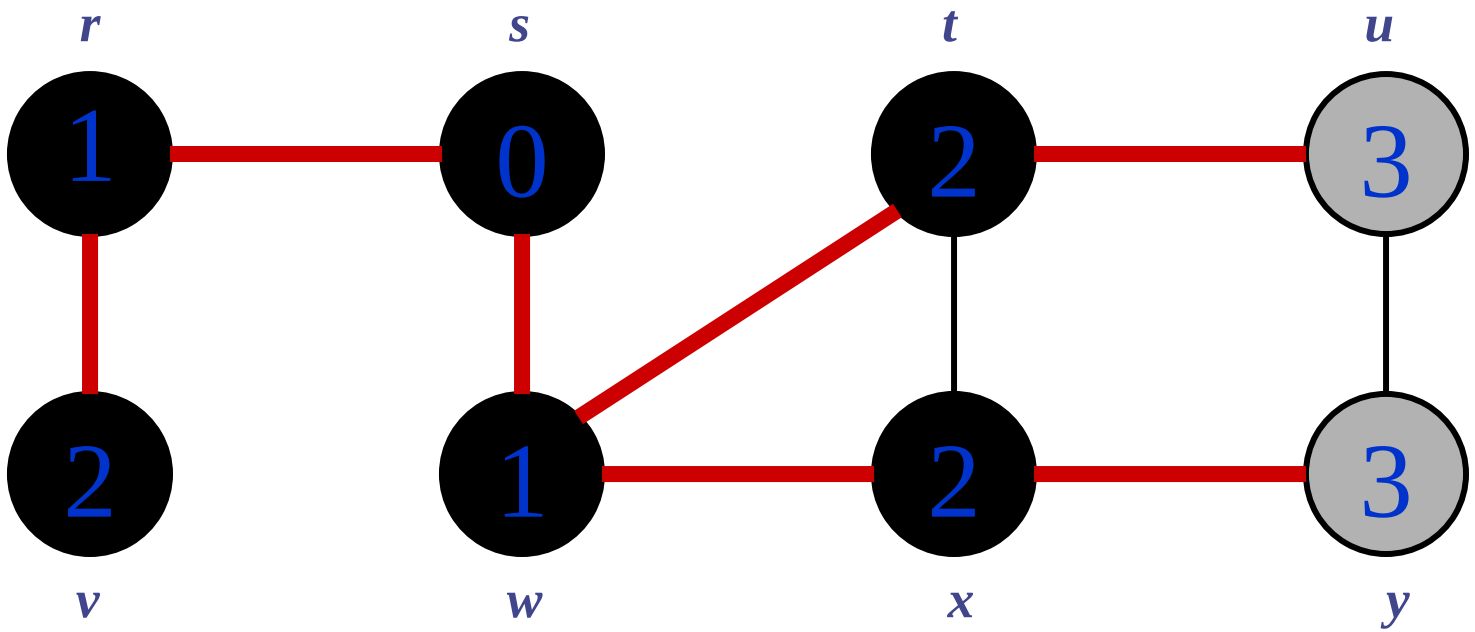
$Q:$ | x | v | u |

Breadth-First Search: Example



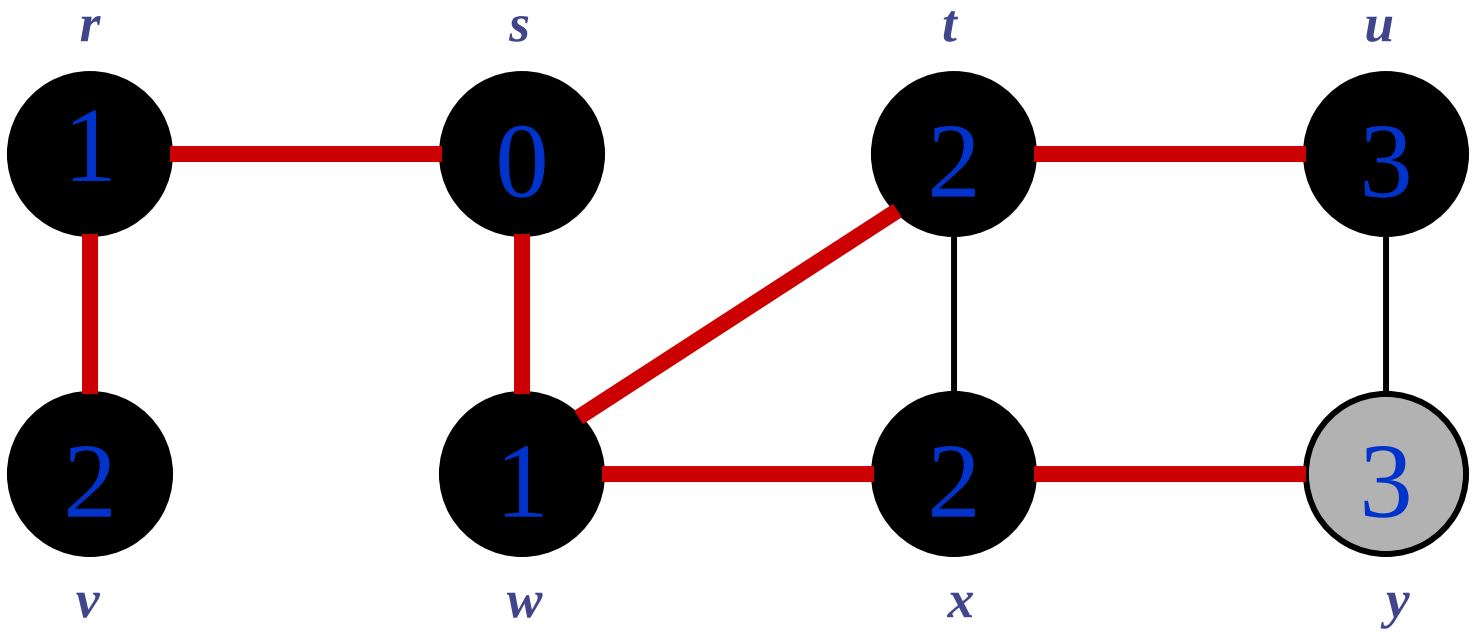


Breadth-First Search: Example



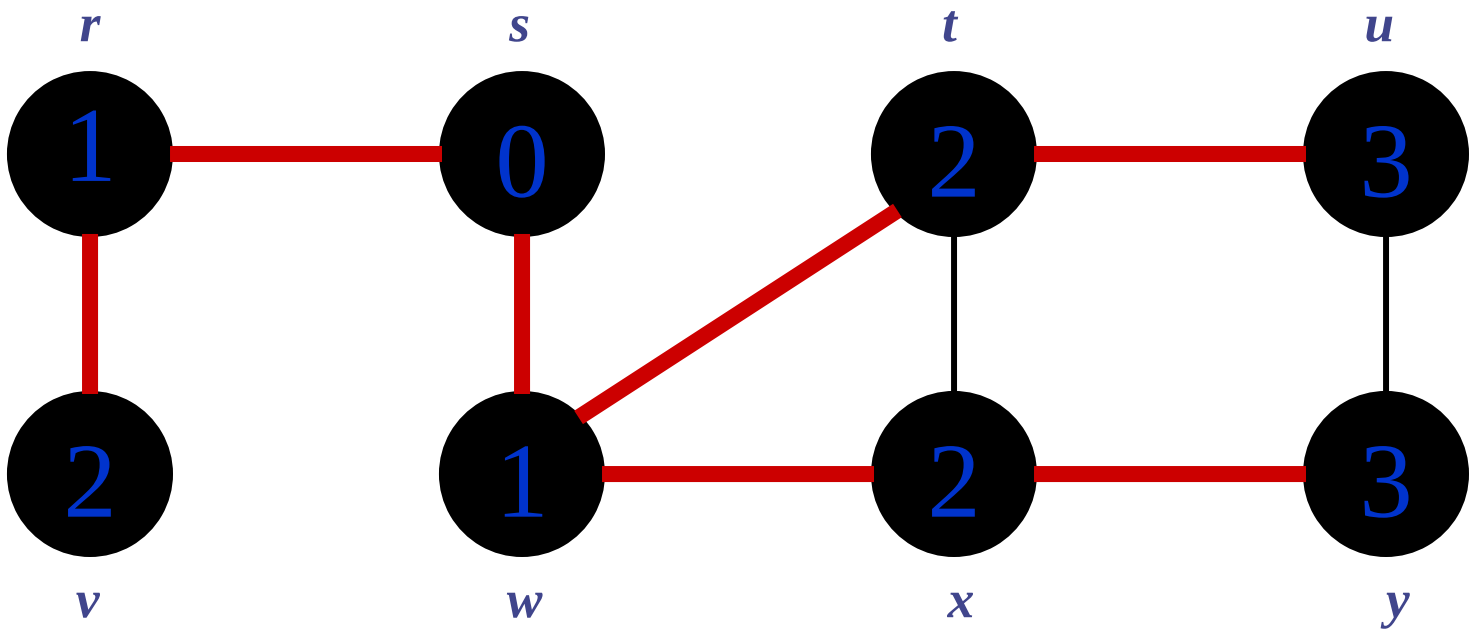
$Q:$ | u | y |

Breadth-First Search: Example



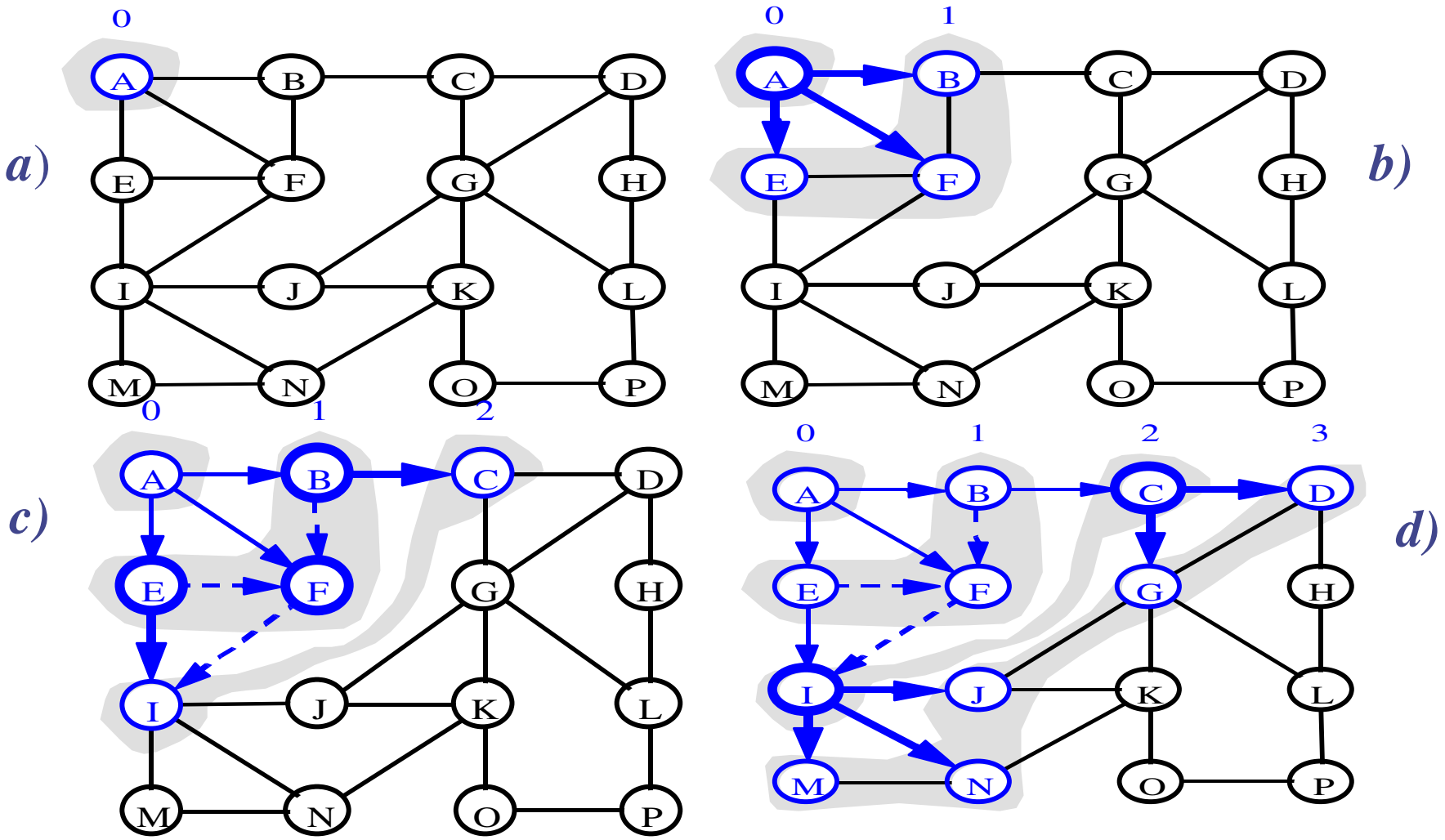
Q: | *y* |

Breadth-First Search: Example

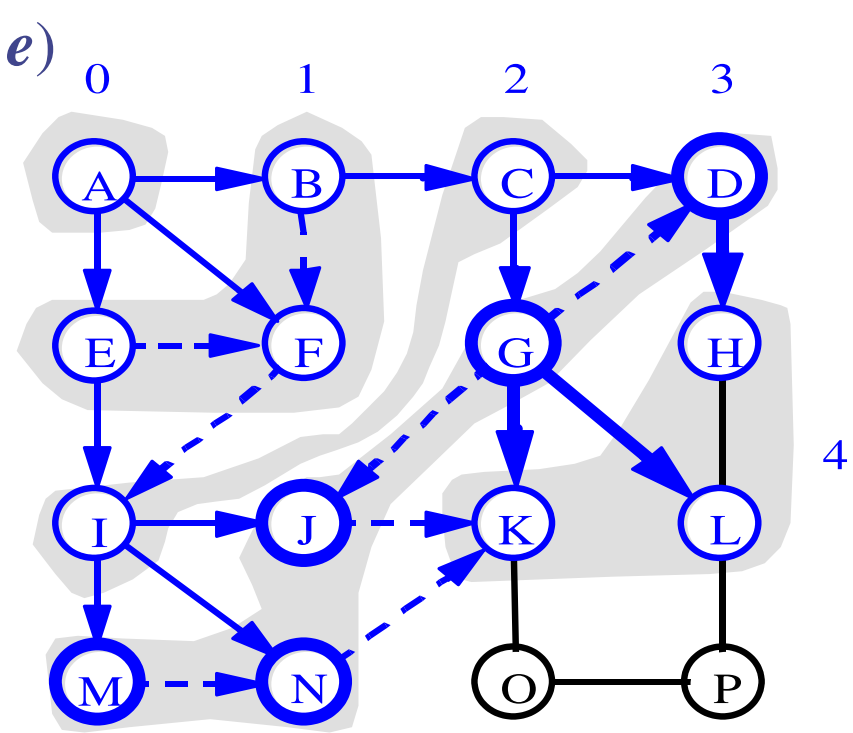


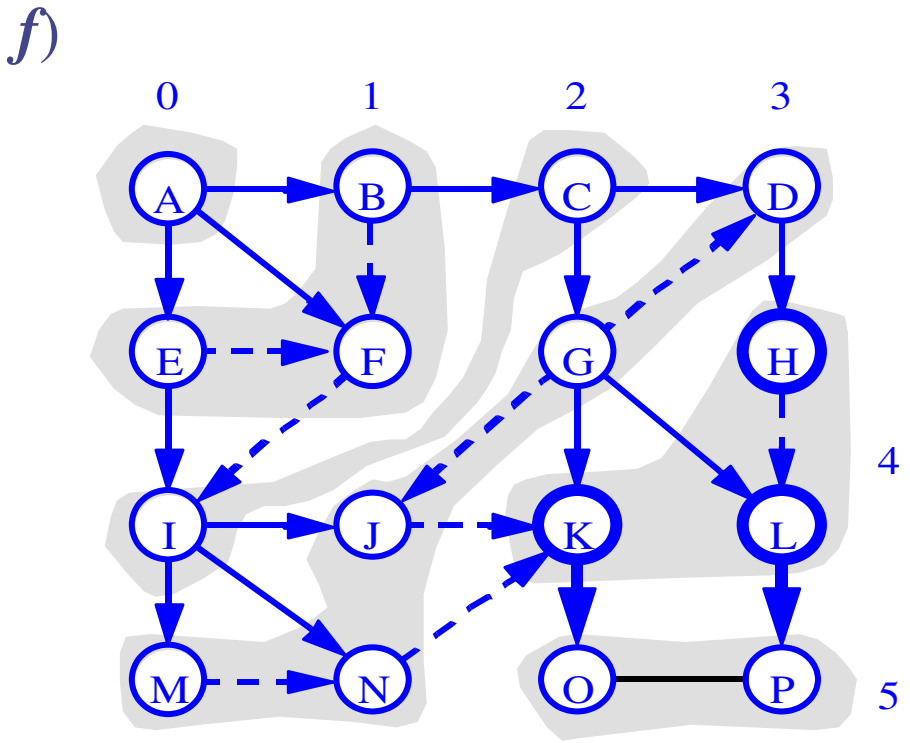
$Q: \quad \emptyset$

BFS - A Graphical Representation



BFS - A Graphical Representation





BFS: The Code Again

```
BFS(G, s) {
    initialize vertices;
    Q = {s};
    while (Q not empty) {
        u = RemoveTop(Q);
        for each v ∈ u->adj {
            if (v->color == WHITE)
                v->color = GREY;
                v->d = u->d + 1;
                v->p = u;
                Enqueue(Q, v);
            }
        u->color = BLACK;
    }
}
```

Touch every vertex: O(V)

u = every vertex, but only once
(Why?)

So v = every vertex that appedrs in some other vert's adjacency list

What will be the running time?

Total running time: O(V+E)

BFS: The Code Again

```
BFS(G, s) {
    initialize vertices;
    Q = {s};
    while (Q not empty) {
        u = RemoveTop(Q);
        for each v ∈ u->adj {
            if (v->color == WHITE)
                v->color = GREY;
                v->d = u->d + 1;
                v->p = u;
                Enqueue(Q, v);
        }
        u->color = BLACK;
    }
}
```

What will be the storage cost in addition to storing the tree?

Total space used:

$O(V + \Sigma(\text{degree}(v))) = O(V+E)$

Breadth-First Search: Properties

- BFS calculates the *shortest-path distance* to the source node
  - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v, or ∞ if v not reachable from s
- BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G
  - Thus can use BFS to calculate shortest path from one vertex to another in O(V+E) time in an unweighted tree

Depth-First Search

- *Depth-first search* is another strategy for exploring a graph
  - Explore "deeper" in the graph whenever possible
  - Edges are explored out of the most recently discovered vertex *v* that still has unexplored edges
  - When all of *v*'s edges have been explored, backtrack to the vertex from which *v* was discovered

- Vertices initially colored white
- Then colored grey when discovered
- Then black when finished

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V{
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V{
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]{
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

What does* u->d *represent?

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

What does* `u->f` *represent?

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

Will all vertices eventually be colored black?

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

What will be the running time?

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

Running time: $O(n^2)$ because call DFS_Visit on each vertex, and the loop over Adj[] can run as many as $|V|$ times

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

BUT, there is actually a tighter bound.

How many times will DFS_Visit() actually be called?

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

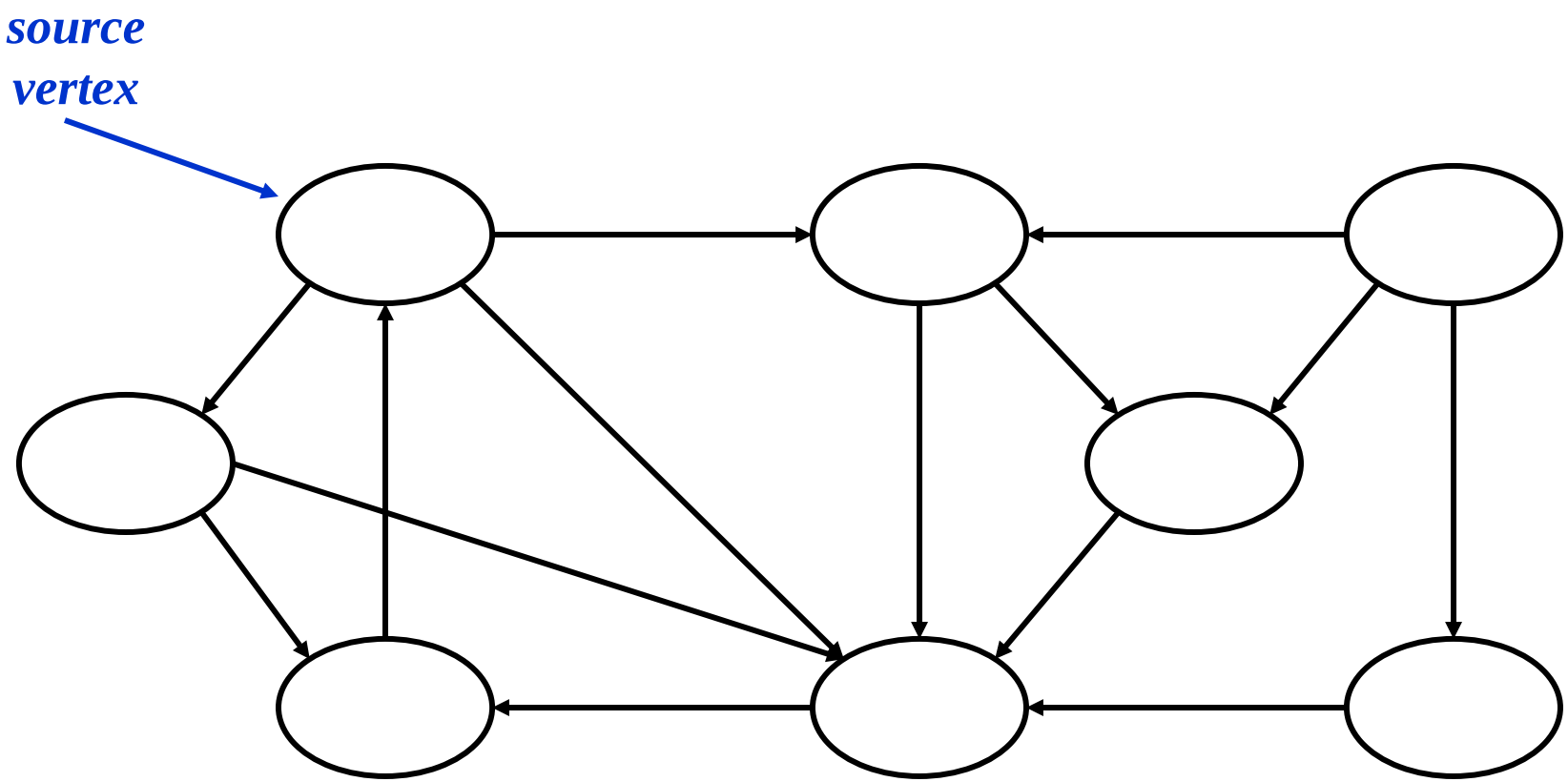
```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

So, running time of DFS = O(V+E)

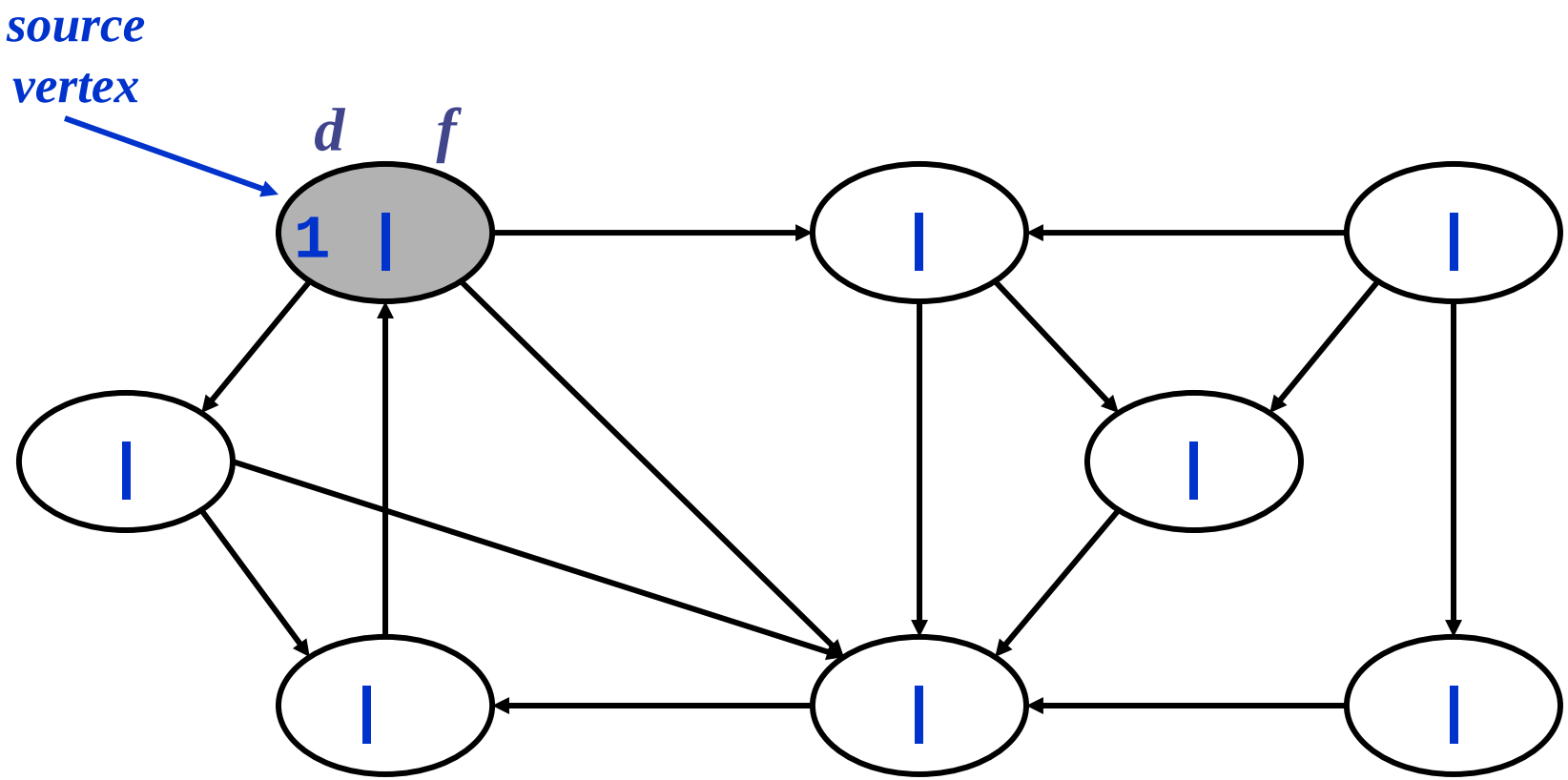
Depth-First Search Analysis

- This running time argument is an informal example of *amortized analysis*
  - "Charge" the exploration of edge to the edge:
    - Each loop in DFS_Visit can be attributed to an edge in the graph
    - Runs once/edge if directed graph, twice if undirected
    - Thus loop will run in O(E) time, algorithm O(V+E)
  - Storage requirement is O(V+E), since adj list requires O(V+E) storage

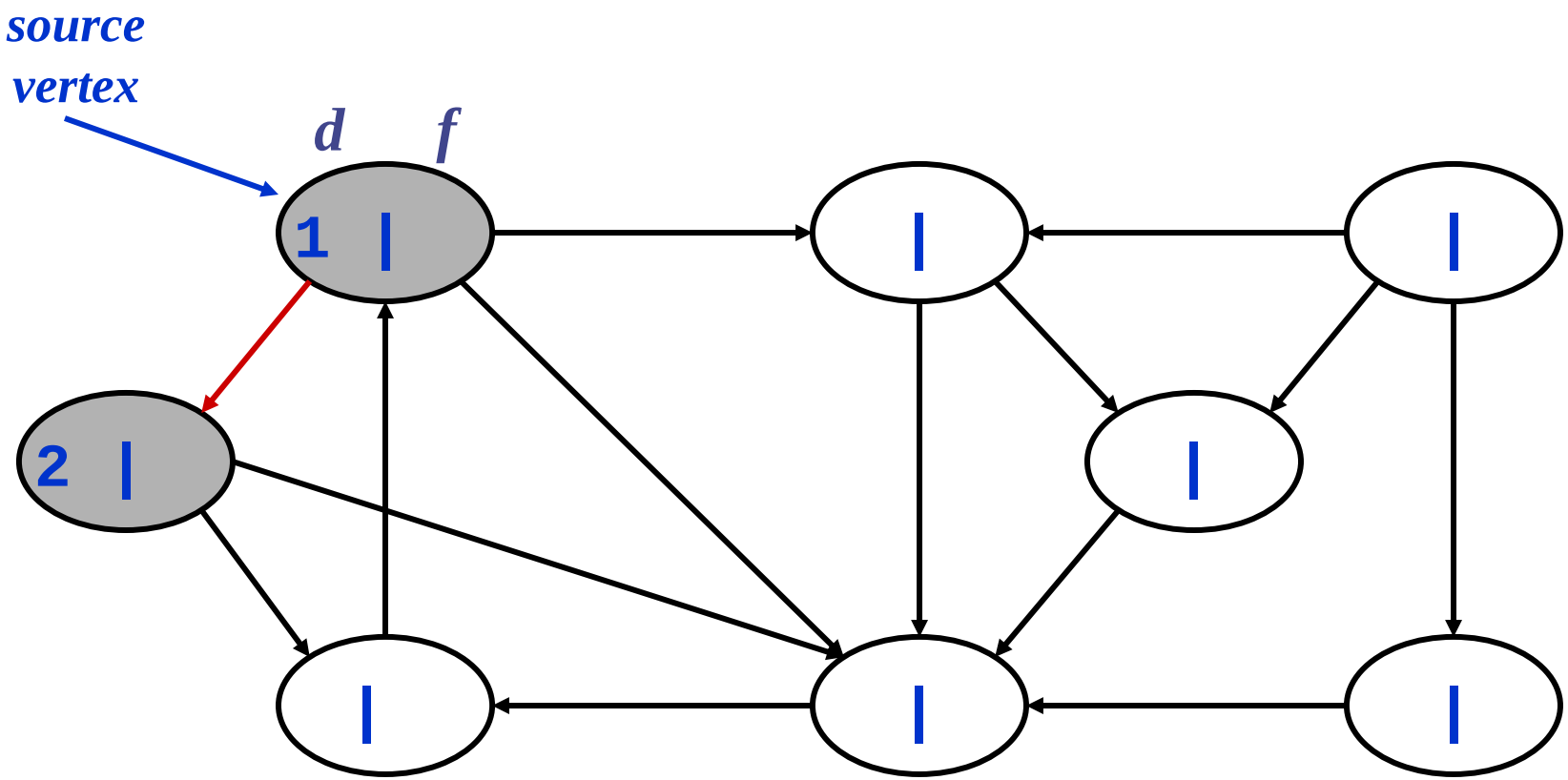
DFS Example



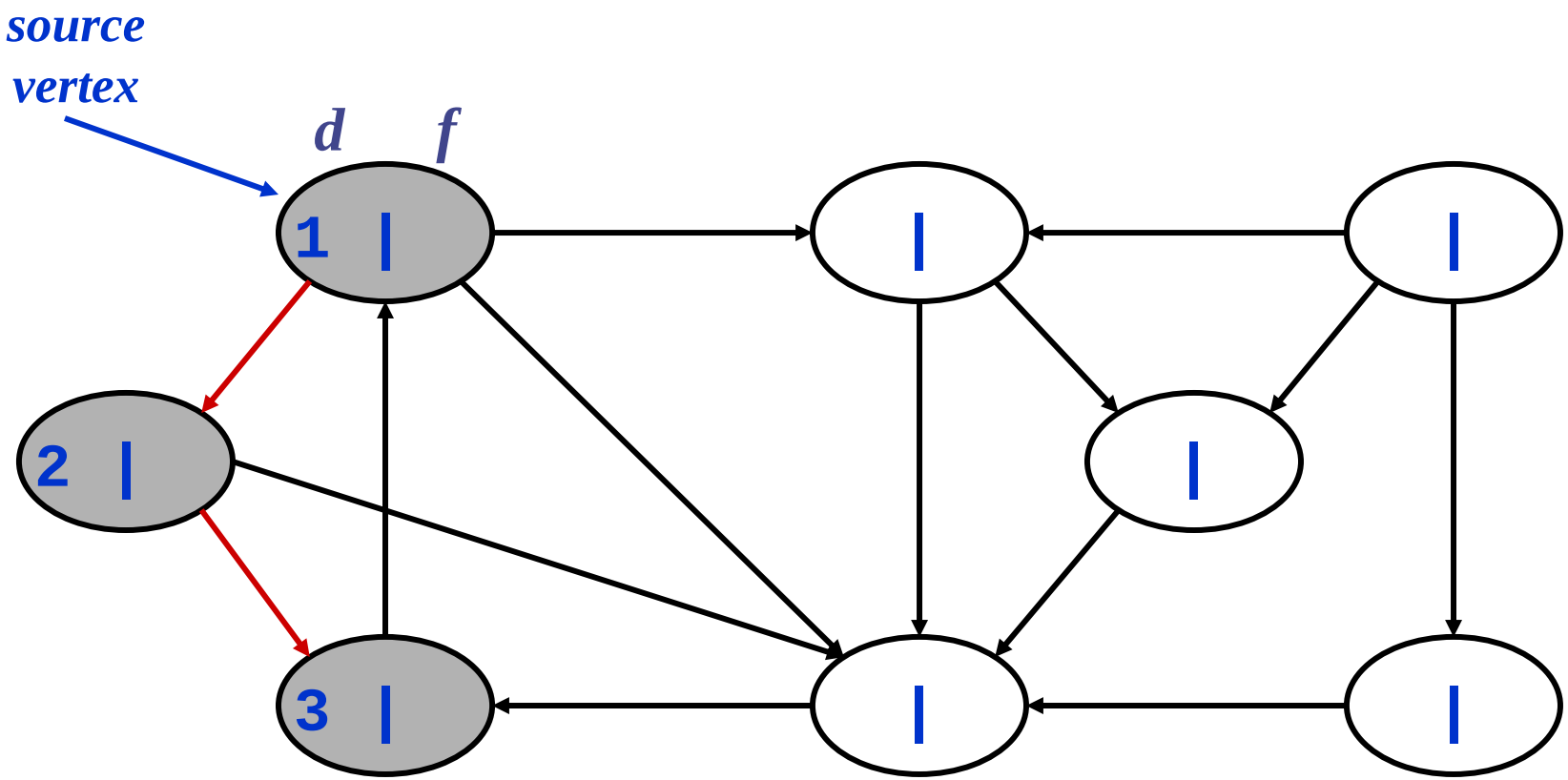
DFS Example



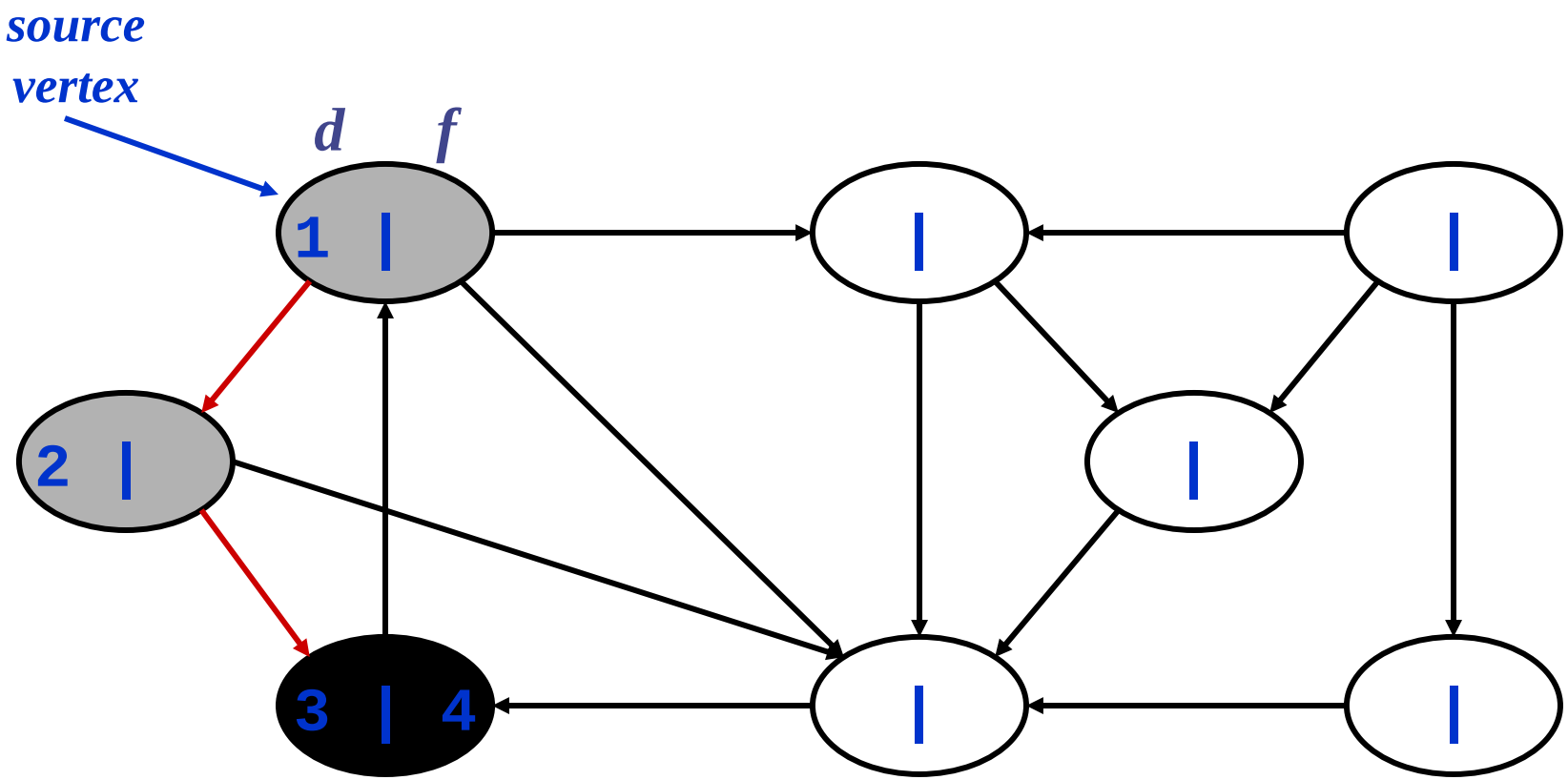
DFS Example



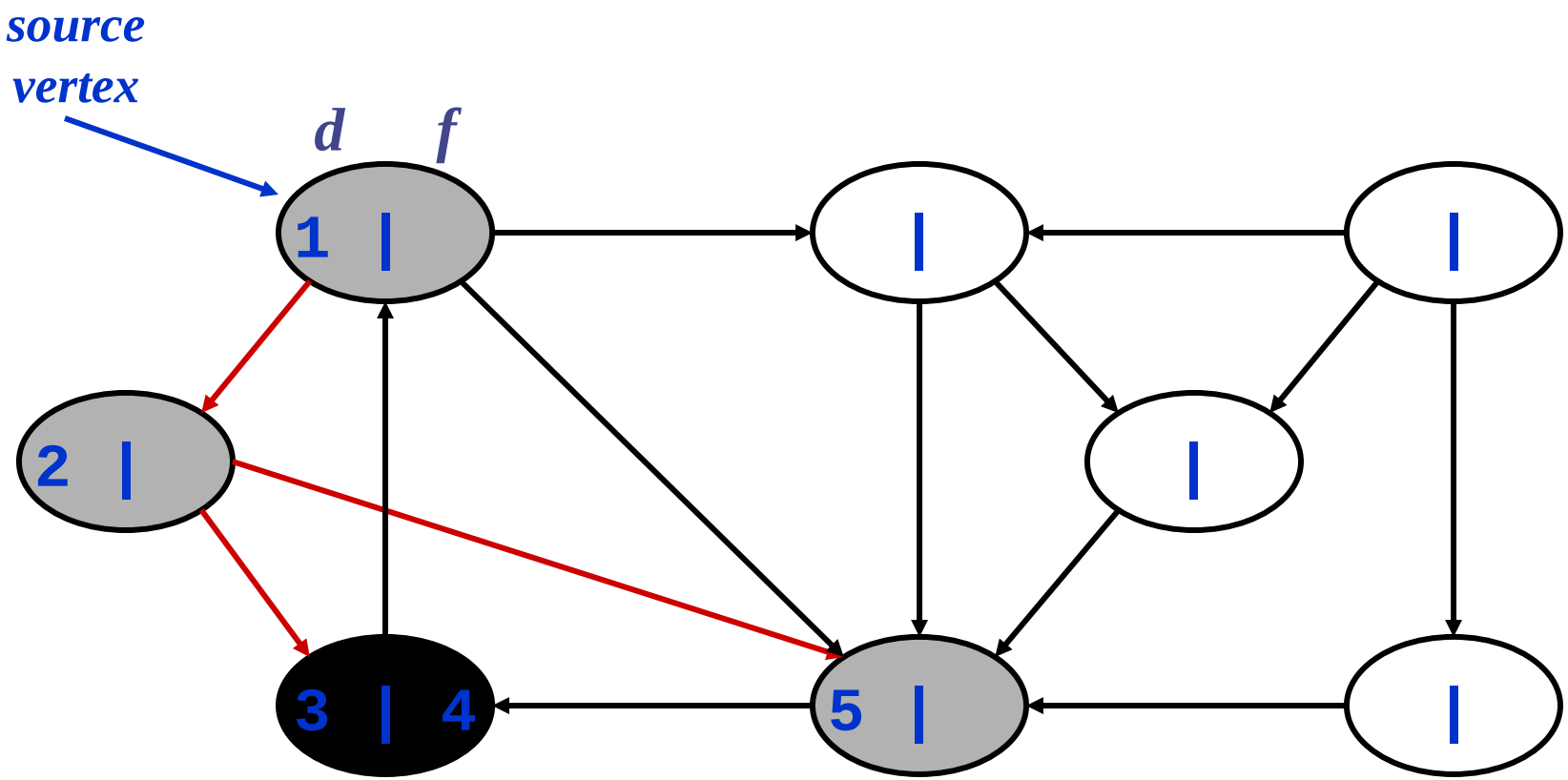
DFS Example



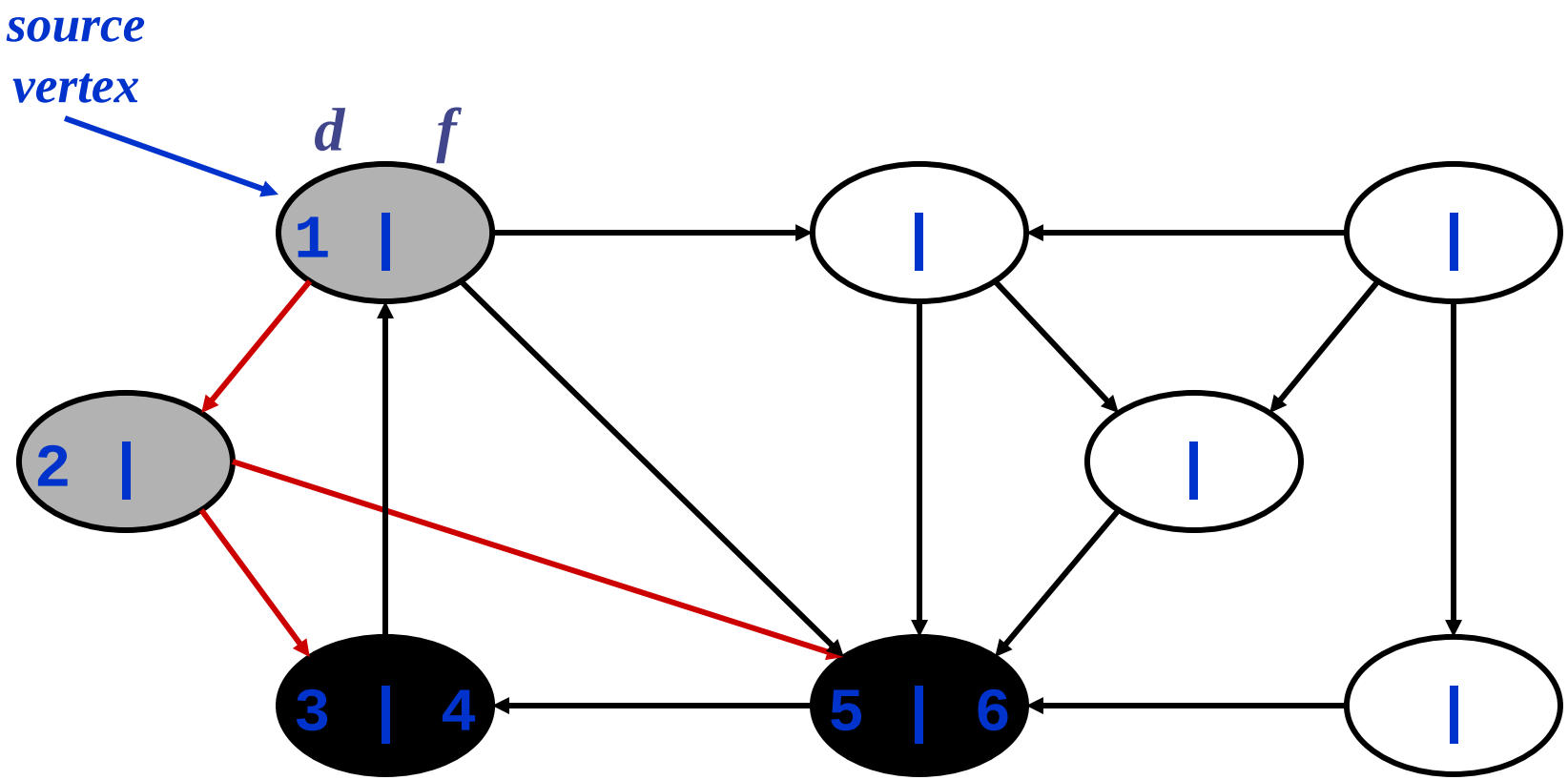
DFS Example



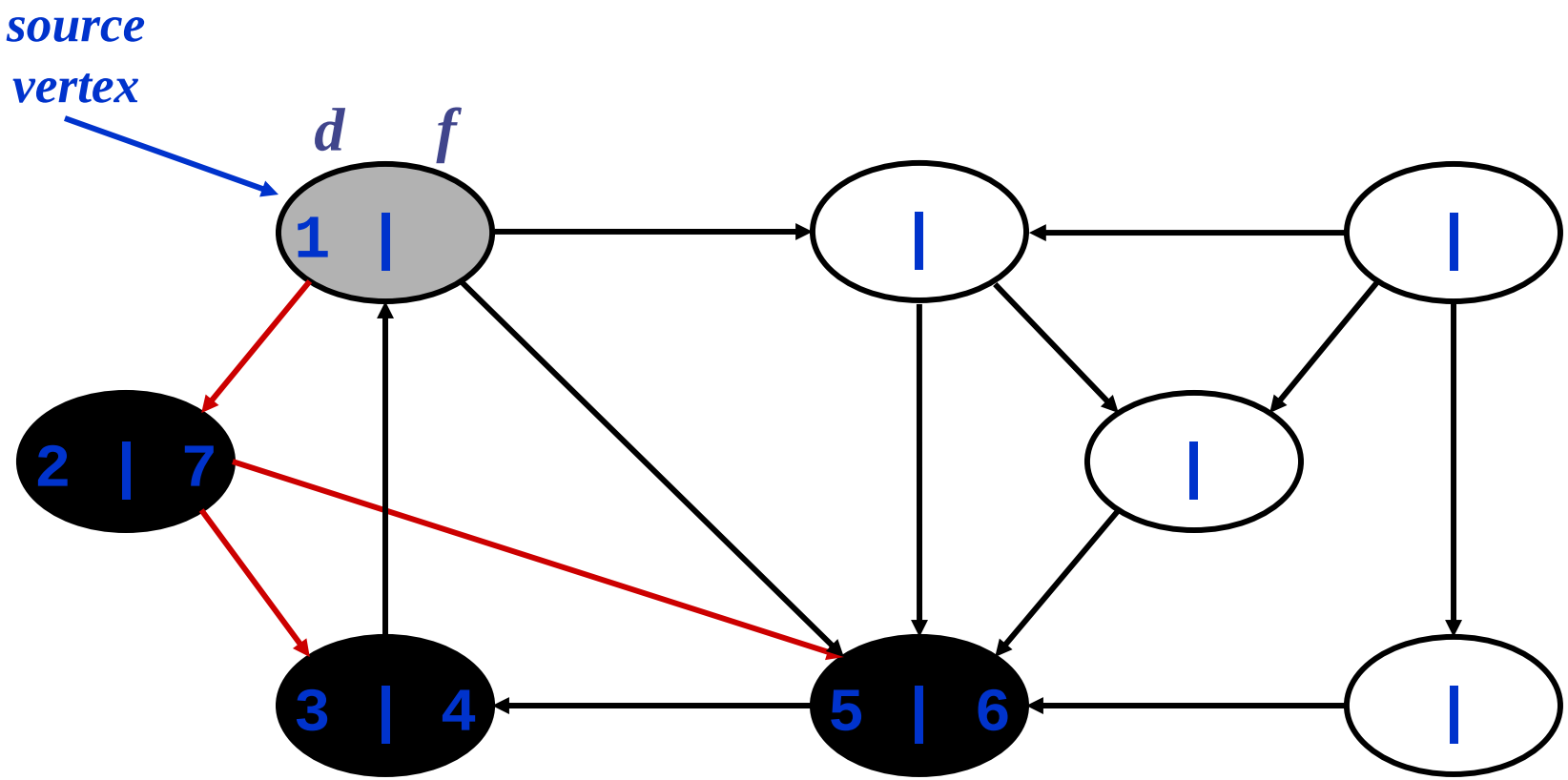
DFS Example



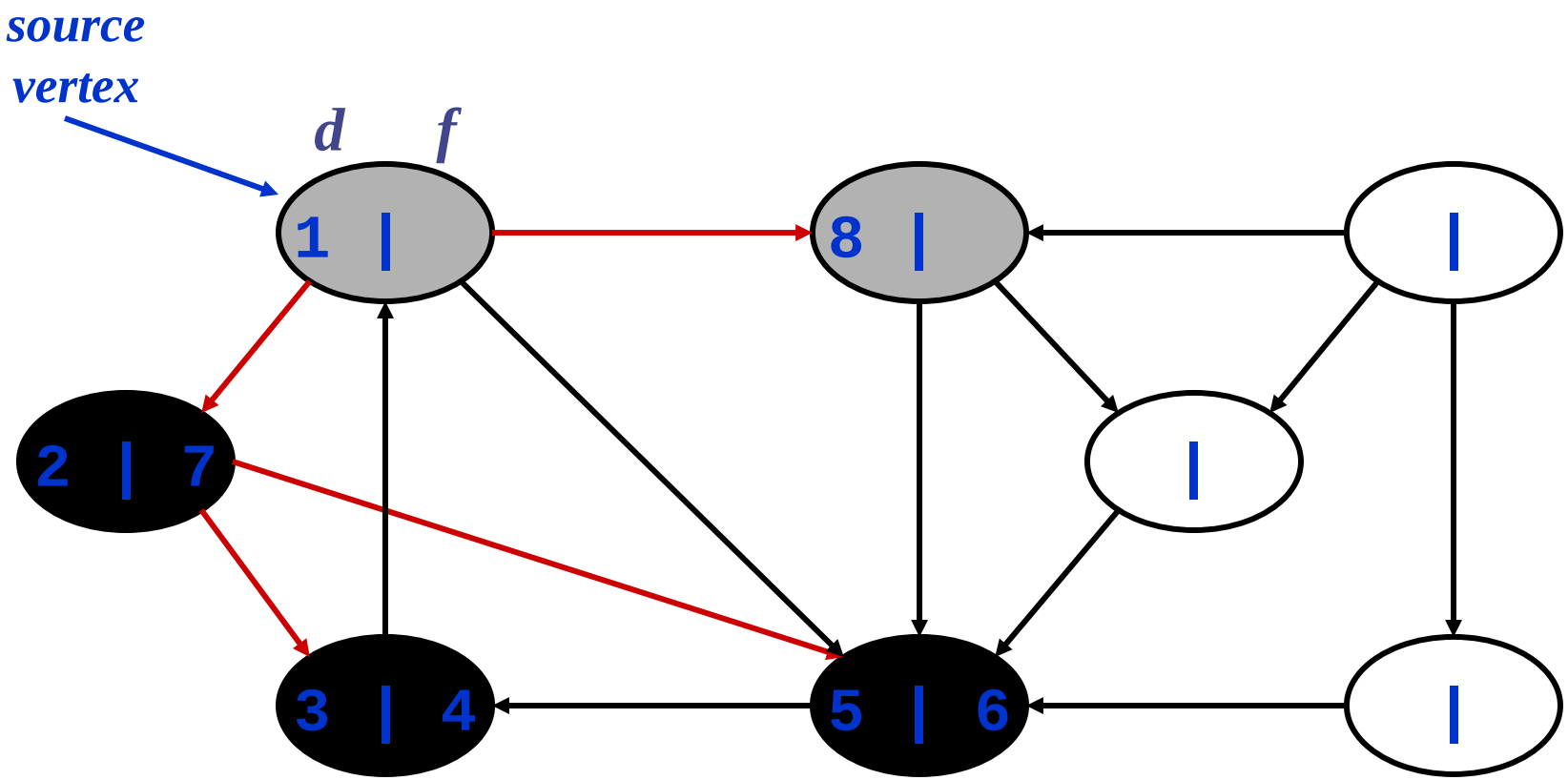
DFS Example



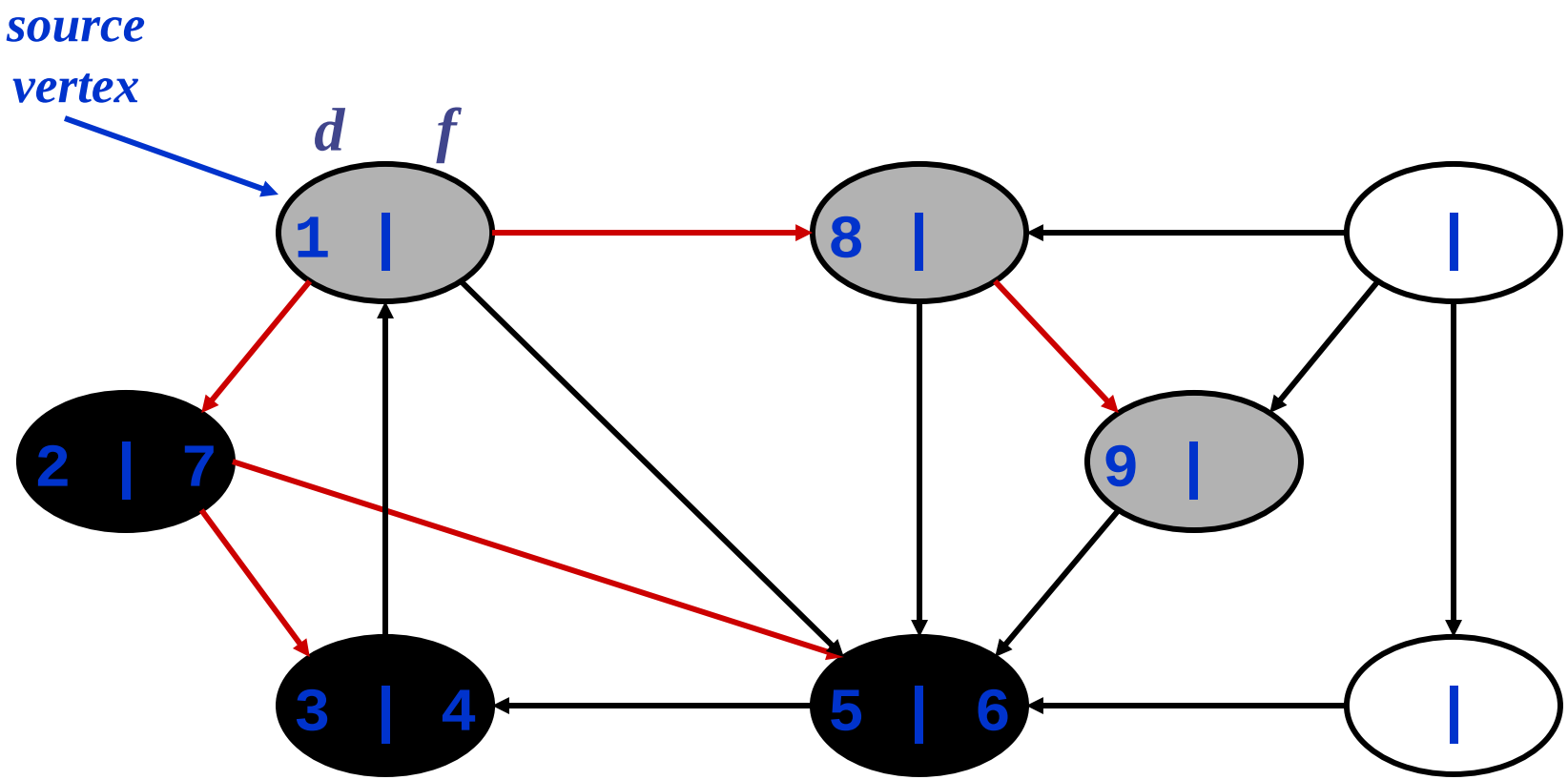
DFS Example



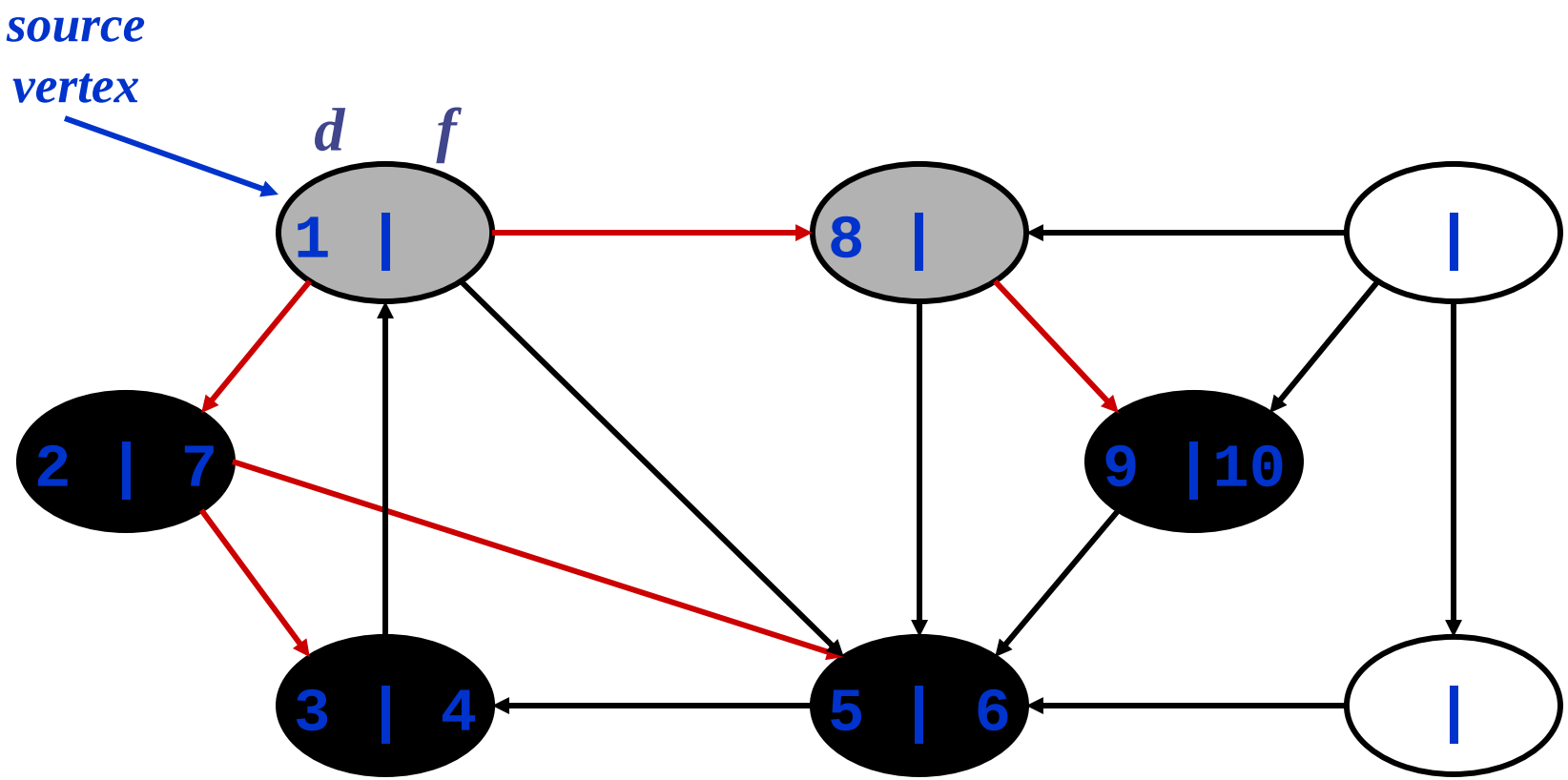
DFS Example



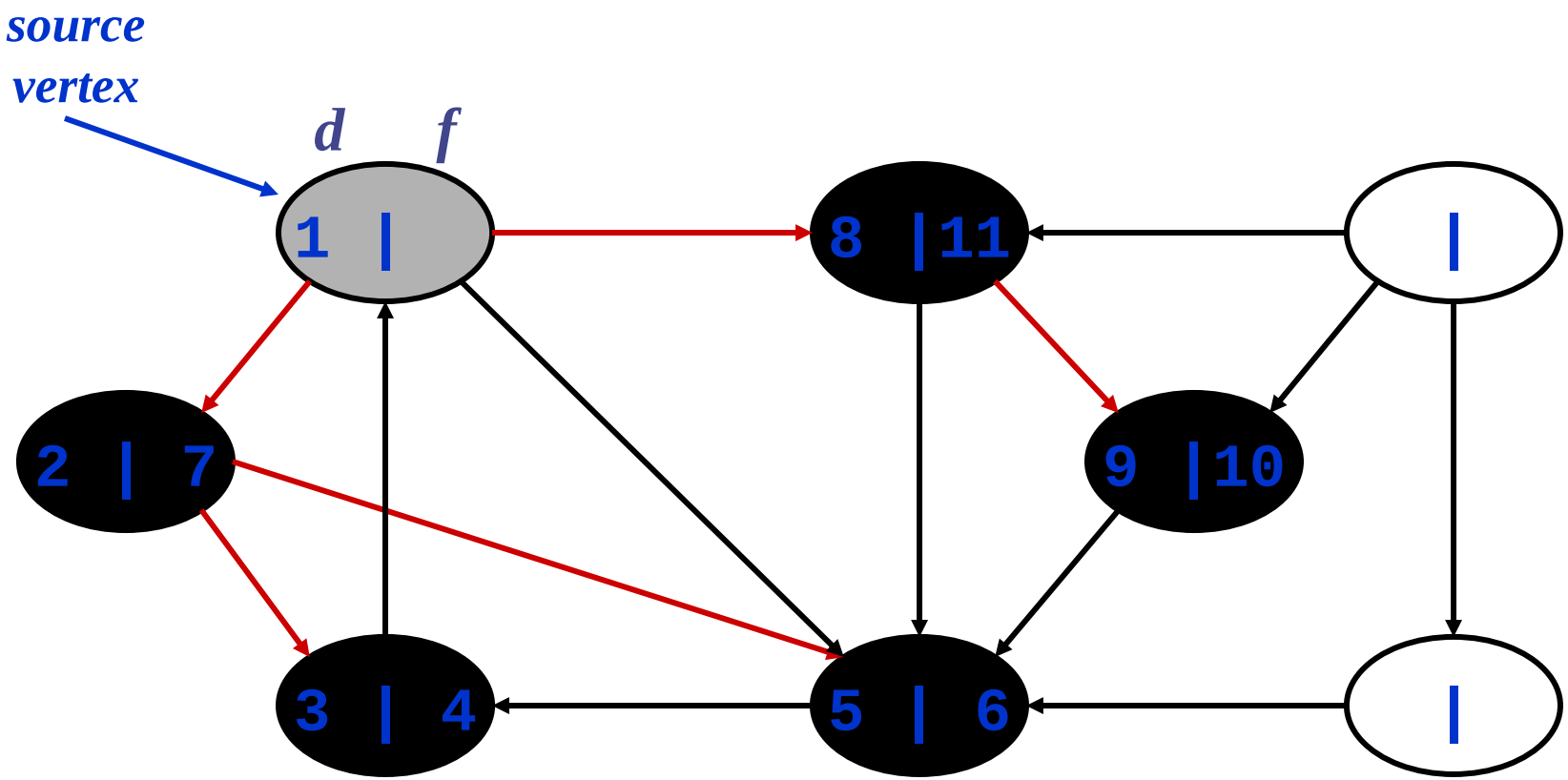
DFS Example



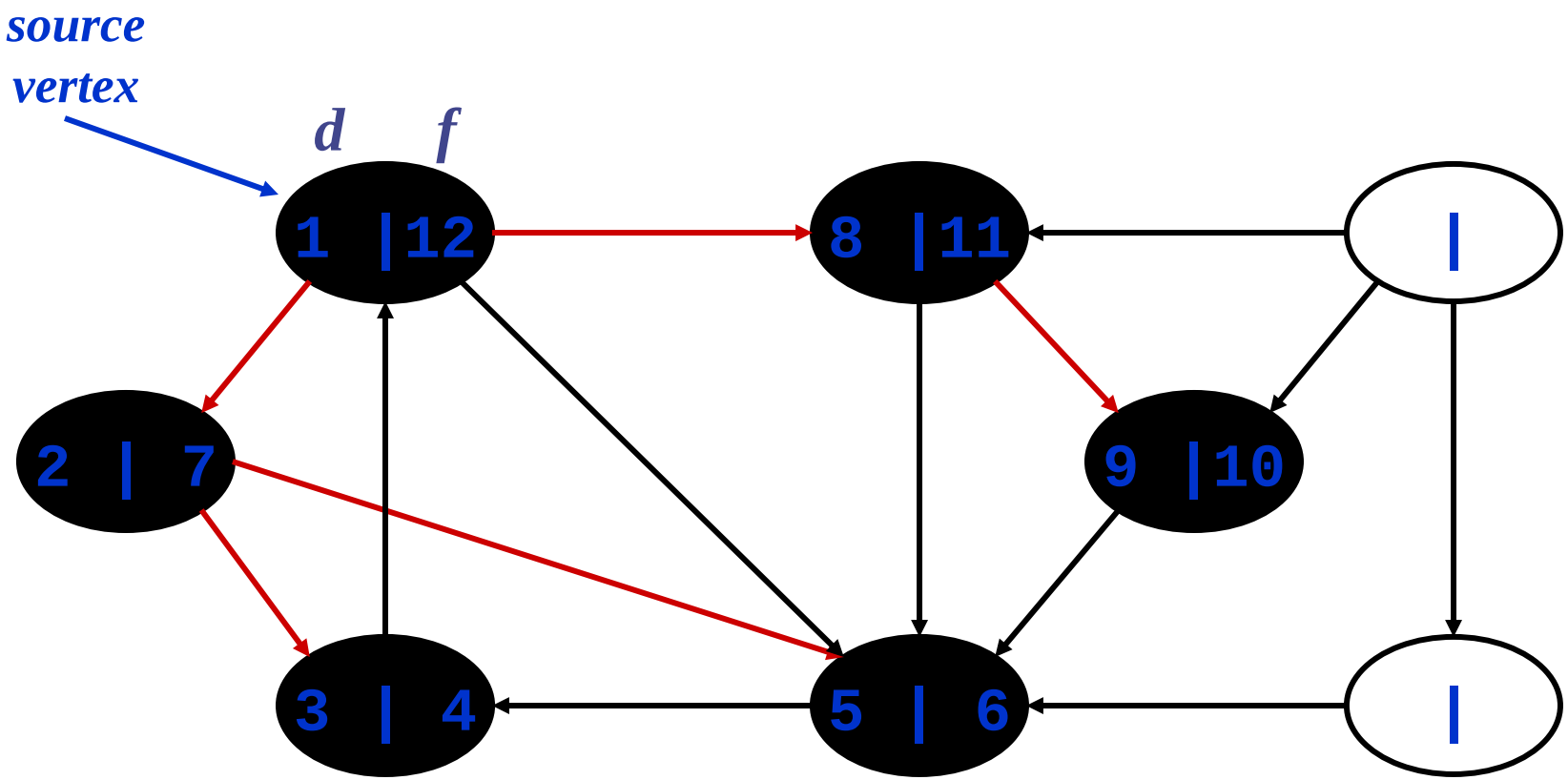
DFS Example



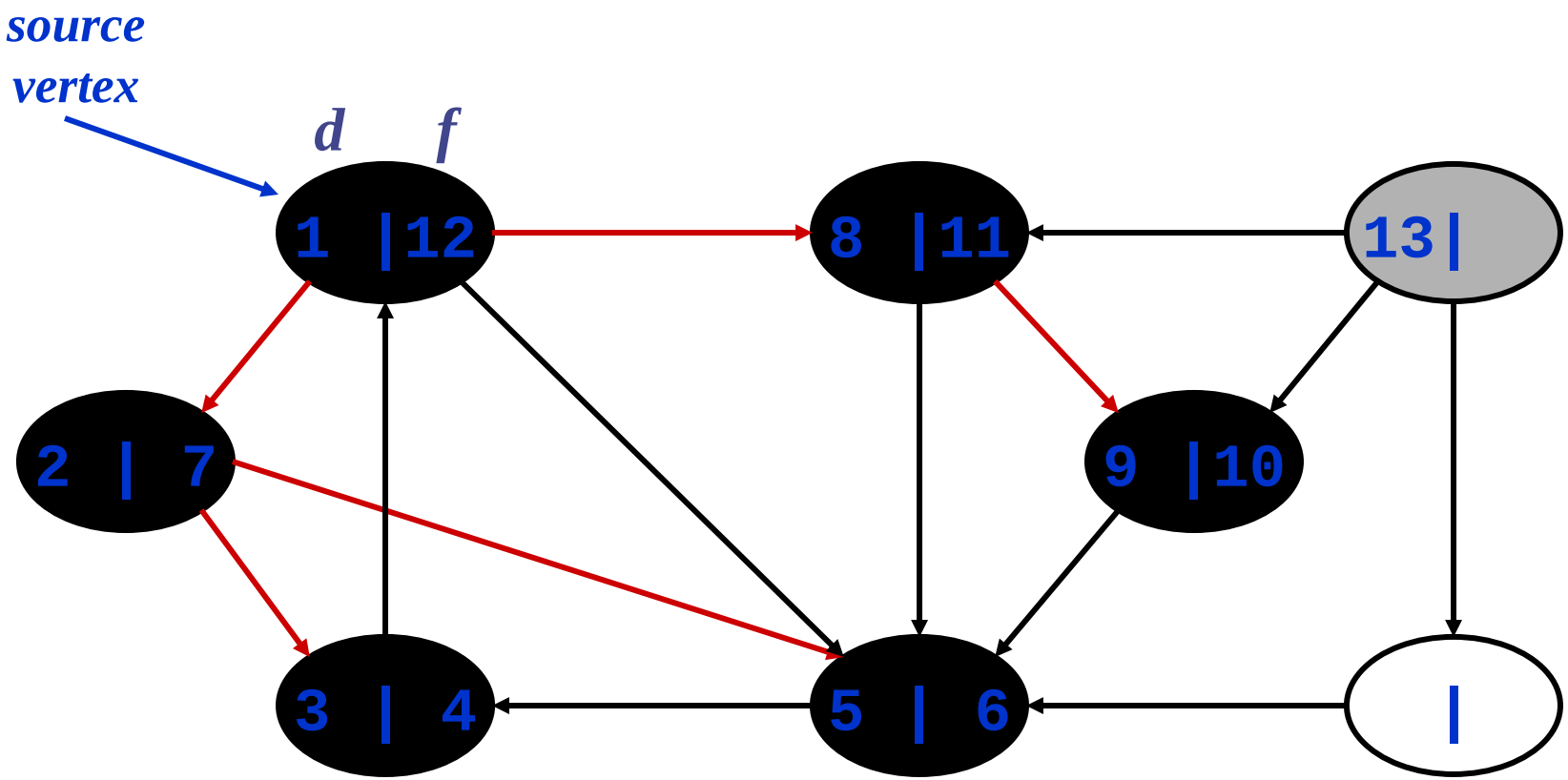
DFS Example



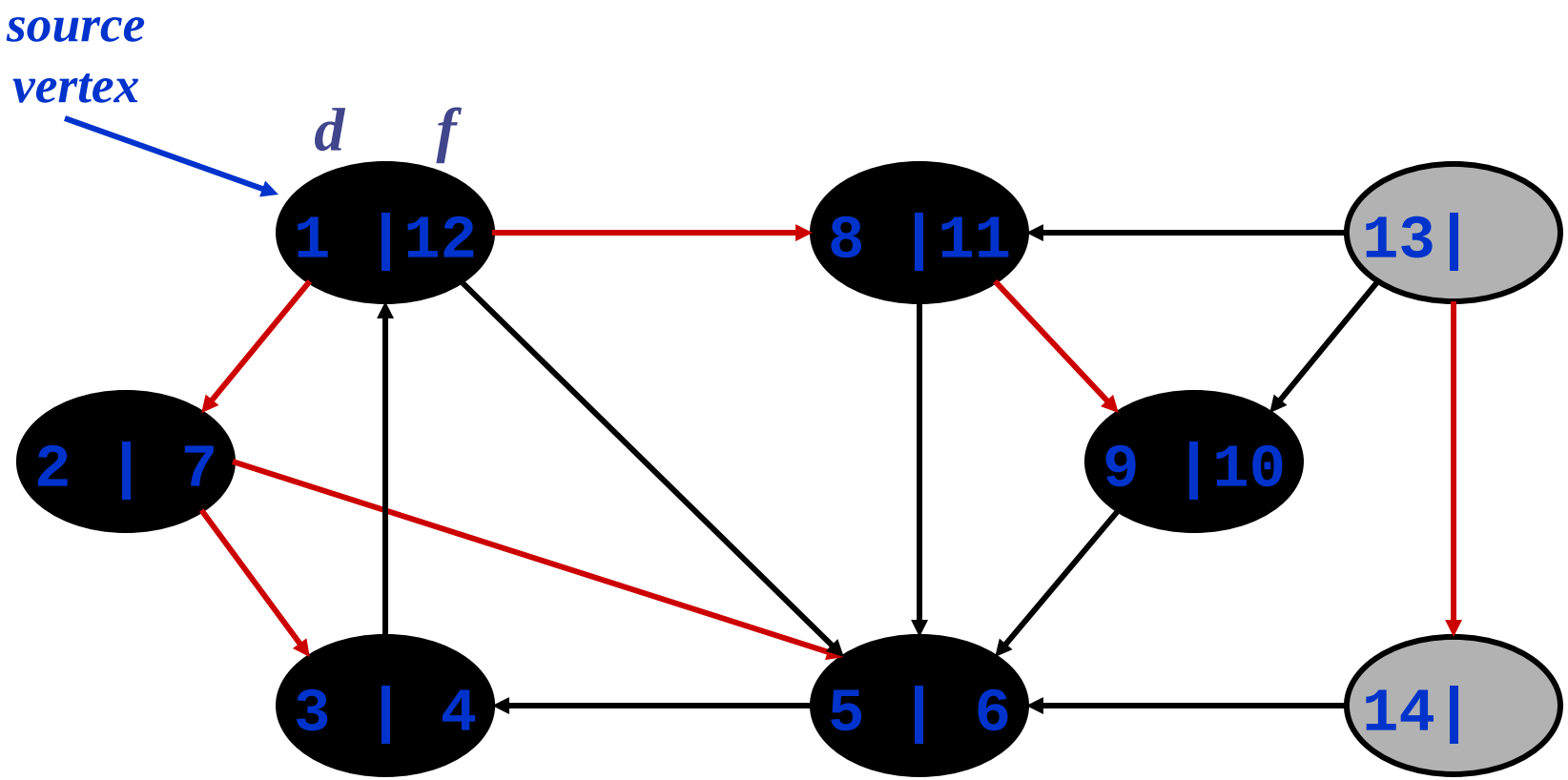
DFS Example



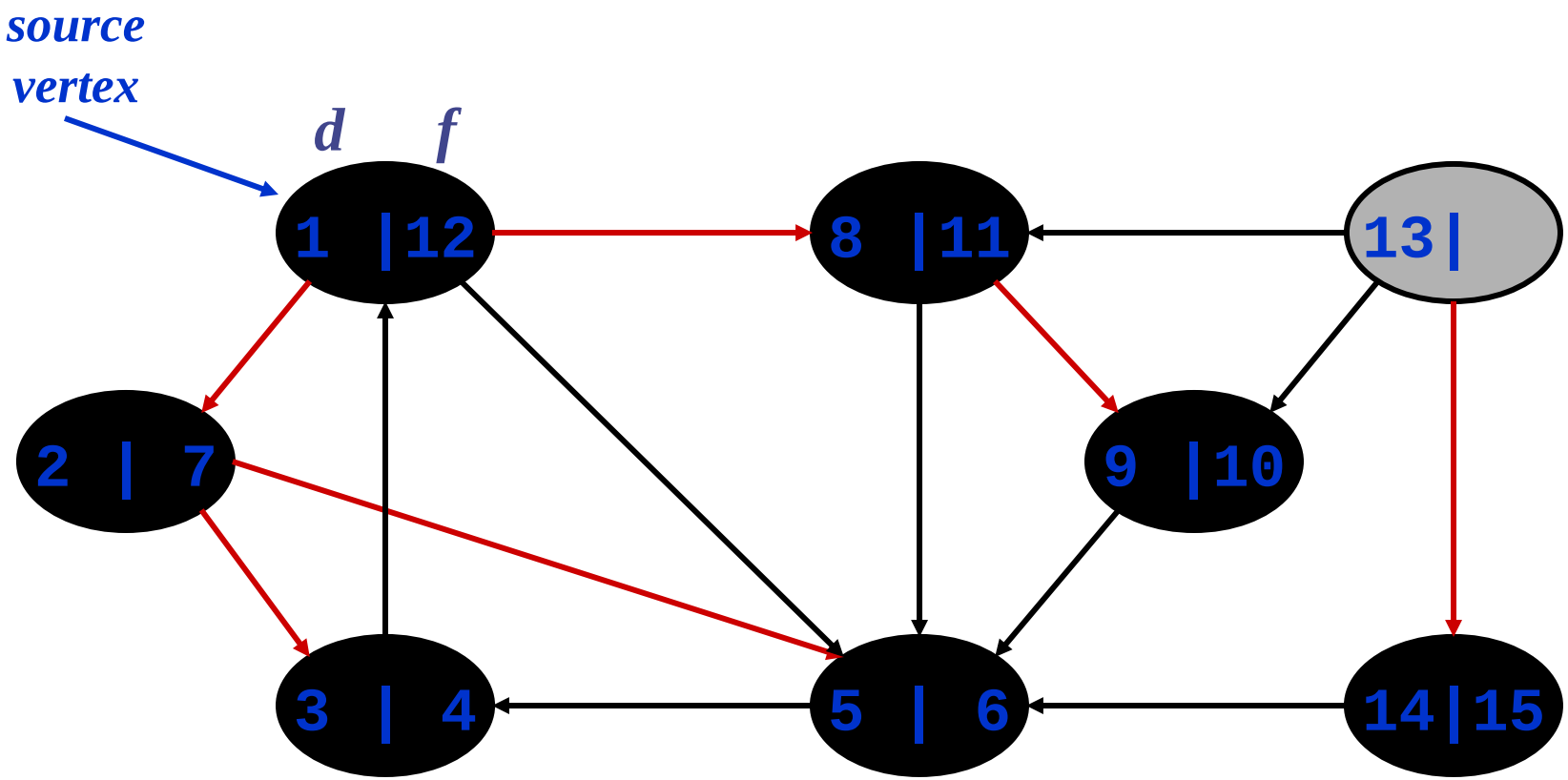
DFS Example



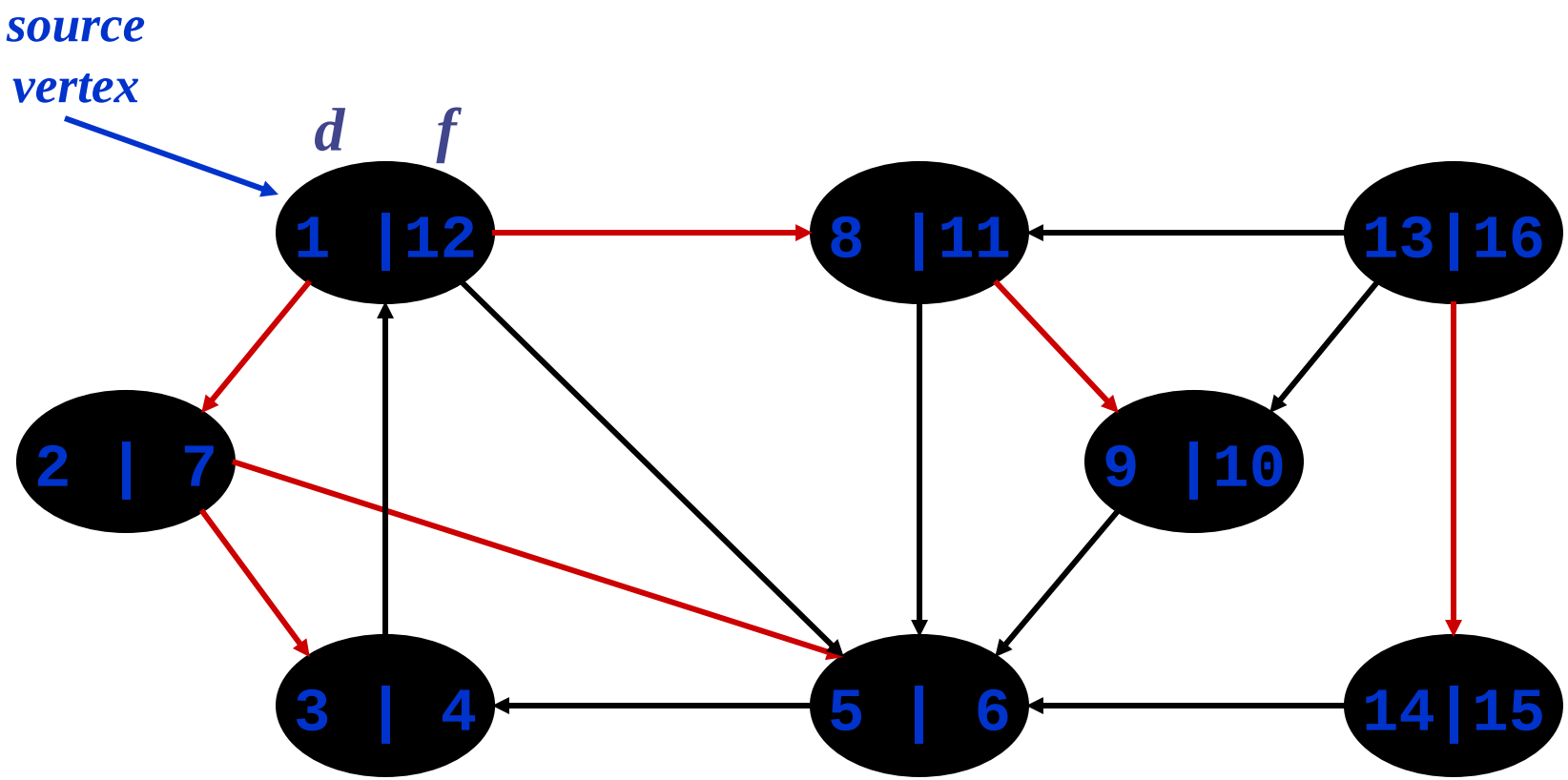
DFS Example



DFS Example



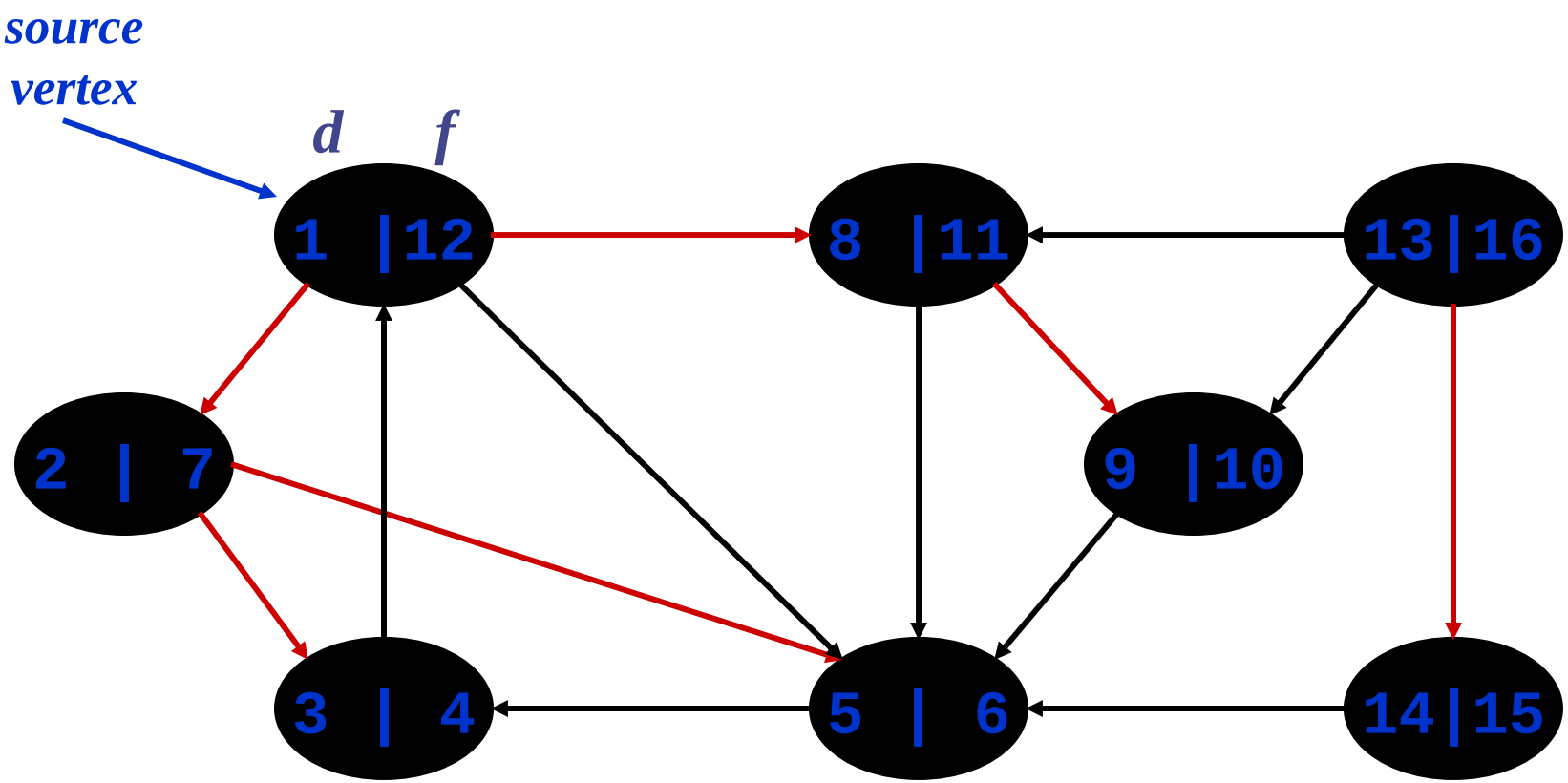
DFS Example



DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
    - The tree edges form a spanning forest
    - *Can tree edges form cycles? Why or why not?*

DFS: Kinds of edges

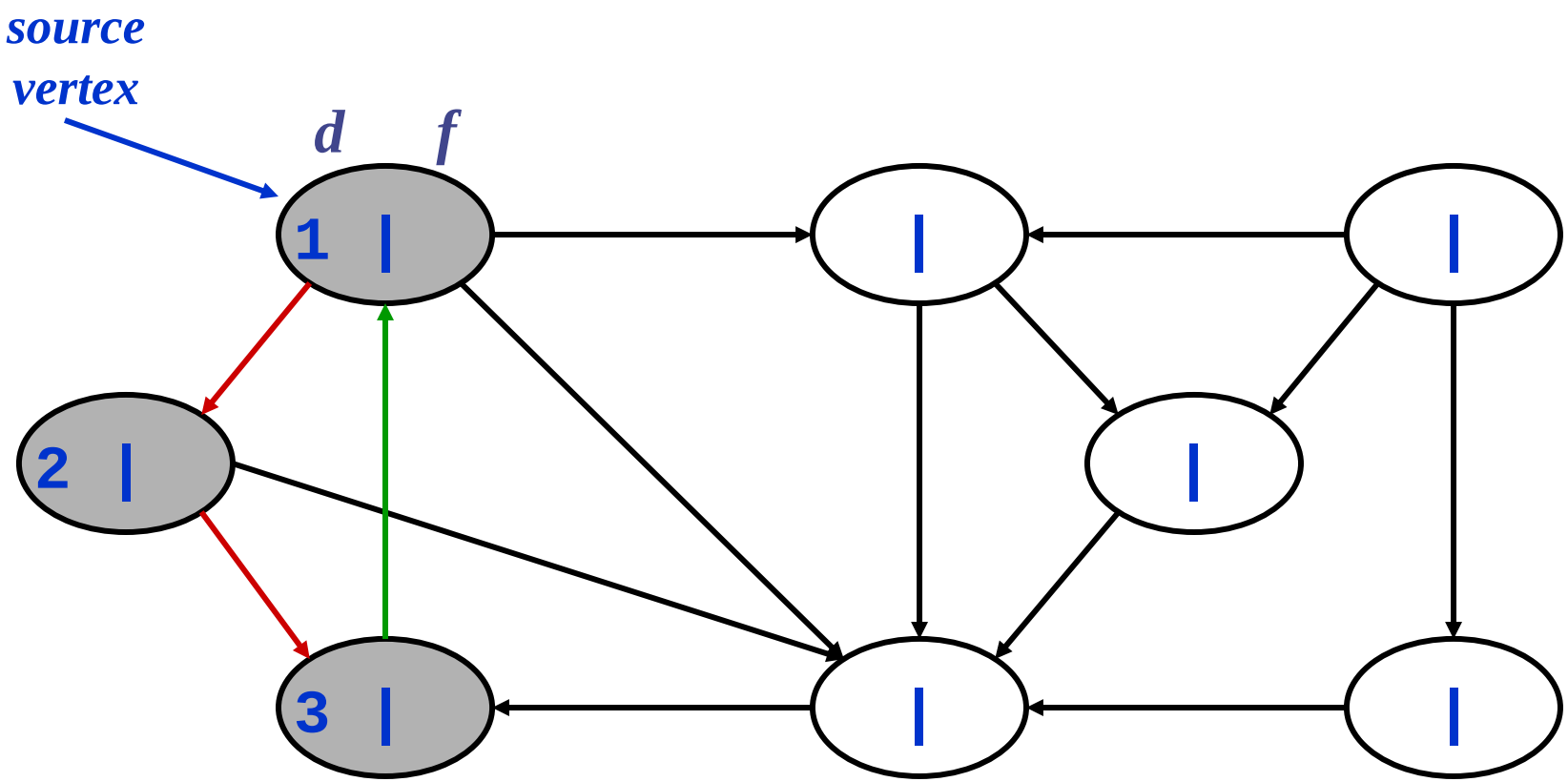


Tree edges

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
    - Encounter a grey vertex (grey to grey)

DFS: Kinds of edges

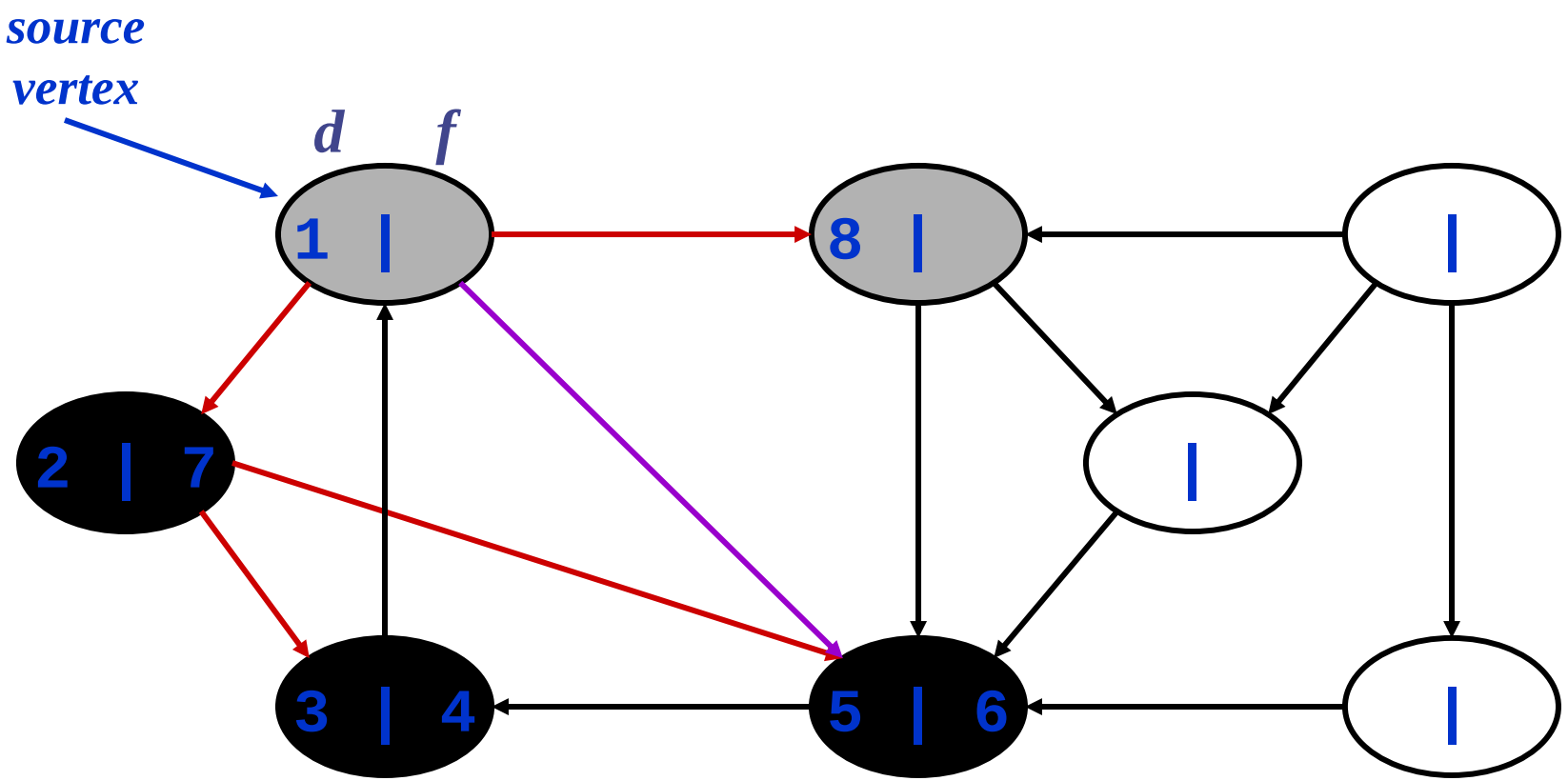


Tree edges *Back edges*

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
  - *Forward edge*: from ancestor to descendent
    - Not a tree edge, though
    - From grey node to black node

DFS: Kinds of edges

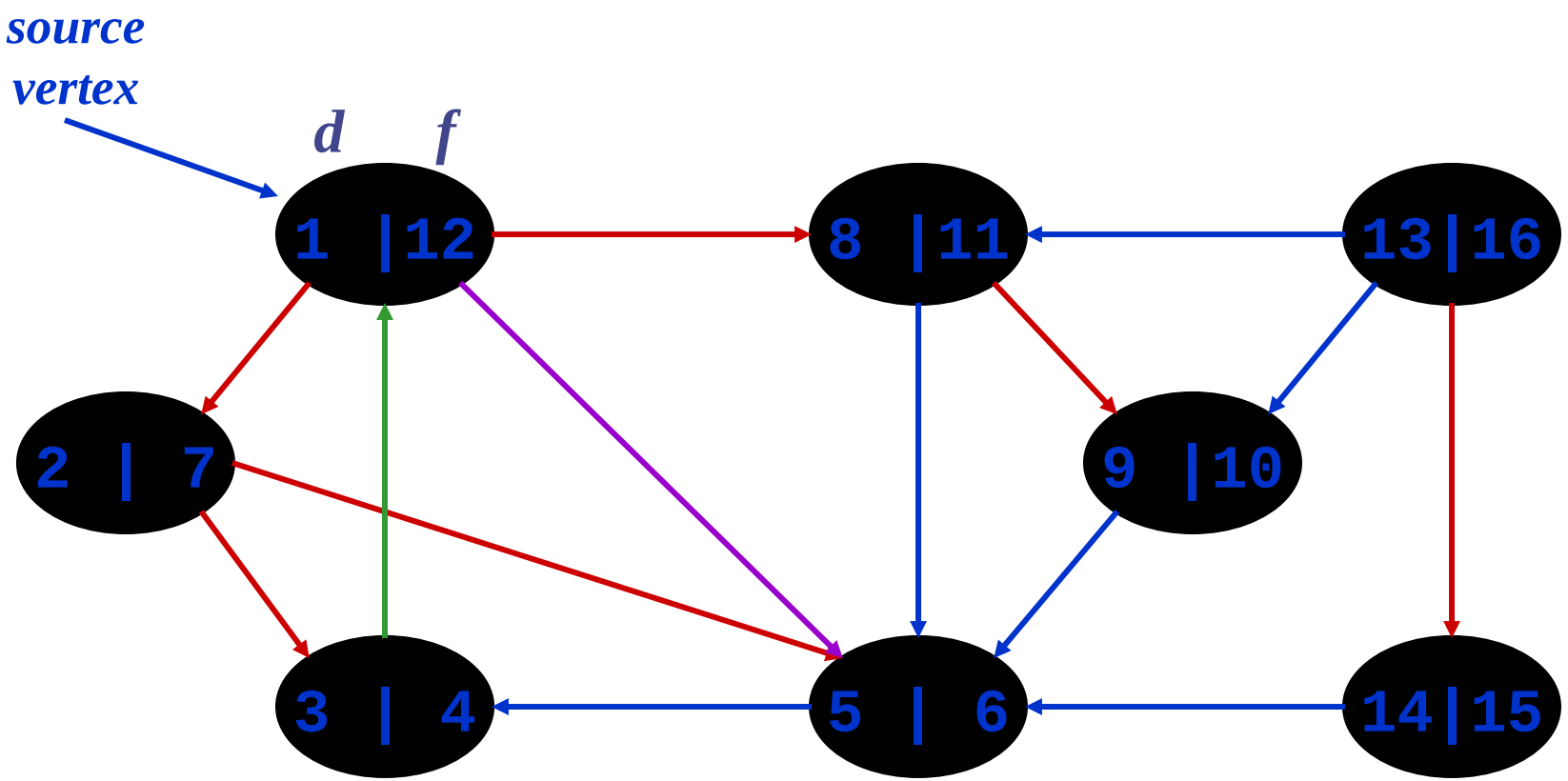


Tree edges *Back edges* *Forward edges*

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
  - *Forward edge*: from ancestor to descendent
  - *Cross edge*: between a tree or subtrees
    - From a grey node to a black node

DFS: Kinds of edges



Tree edges *Back edges* *Forward edges* *Cross edges*

DFS: Kinds of edges

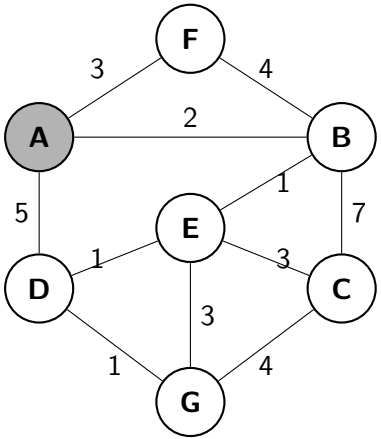
- DFS introduces an important distinction among edges in the original graph:
  - *Tree edge*: encounter new (white) vertex
  - *Back edge*: from descendent to ancestor
  - *Forward edge*: from ancestor to descendent
  - *Cross edge*: between a tree or subtrees

- Note: tree and back edges are very important; some algorithms use forward and cross edges

Dijkstra's Algorithm from Source A – Initial State

Node	Dist(A, Node)	Prev
A	0	–
B	∞	–
C	∞	–
D	∞	–
E	∞	–
F	∞	–
G	∞	–

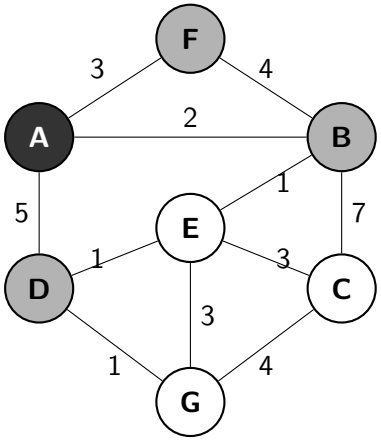
Priority Queue
(A, 0)



Step 1: Extract A, Relax Neighbors B, F, D

Node	Dist(A, Node)	Prev
A	0	–
B	2	A
C	∞	–
D	5	A
E	∞	–
F	3	A
G	∞	–

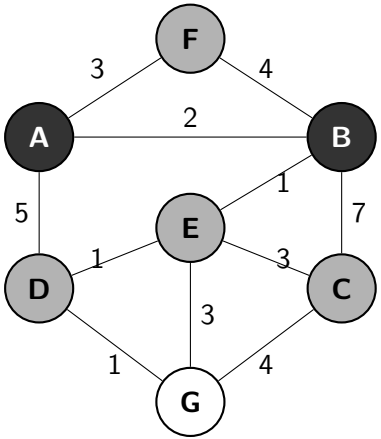
Priority Queue
(B, 2)
(F, 3)
(D, 5)



Step 2: Extract B, Relax C and E

Node	Dist(A, Node)	Prev
A	0	–
B	2	A
C	9	B
D	5	A
E	3	B
F	3	A
G	∞	–

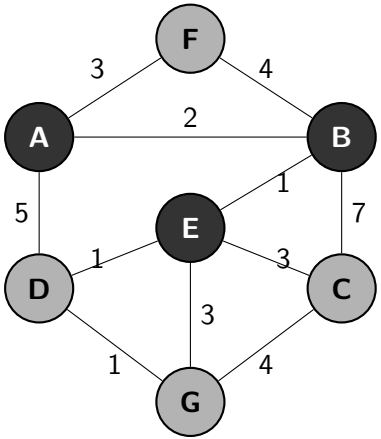
Priority Queue
(E, 3)
(F, 3)
(D, 5)
(C, 9)



Step 3: Extract E, Relax C, D, G

Node	Dist(A, Node)	Prev
A	0	–
B	2	A
C	~~9~~ 6	~~B~~ E
D	~~5~~ 4	~~A~~ E
E	3	B
F	3	A
G	6	E

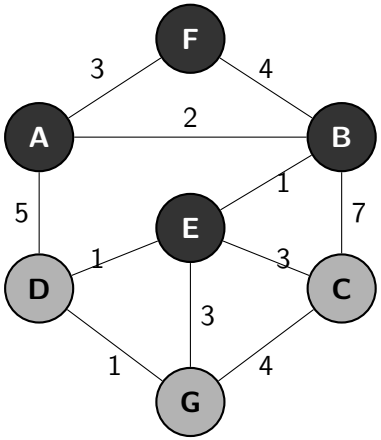
Priority Queue
(F, 3)
(D, 4)
(C, 6)
(G, 6)



Step 4: Extract F (no distance updates)

Node	Dist(A, Node)	Prev
A	0	–
B	2	A
C	~~9~~ 6	~~B~~ E
D	~~5~~ 4	~~A~~ E
E	3	B
F	3	A
G	6	E

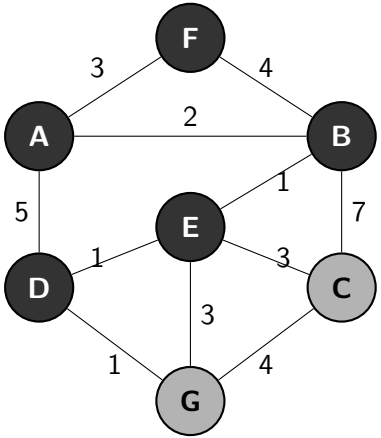
Priority Queue
(D, 4)
(C, 6)
(G, 6)



Step 5: Extract D, Relax G

Node	Dist(A, Node)	Prev
A	0	–
B	2	A
C	~~9~~ 6	~~B~~ E
D	~~5~~ 4	~~A~~ E
E	3	B
F	3	A
G	~~6~~ 5	~~E~~ D

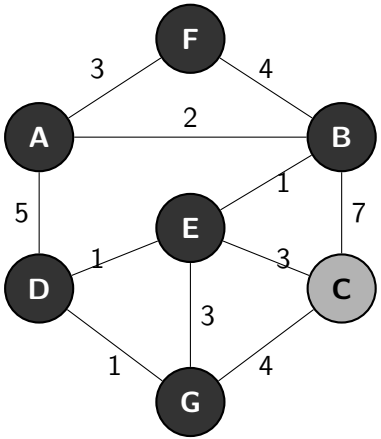
Priority Queue
(G, 5)
(C, 6)



Step 6: Extract G (no distance updates)

Node	Dist(A, Node)	Prev
A	0	–
B	2	A
C	~~9~~ 6	~~B~~ E
D	~~5~~ 4	~~A~~ E
E	3	B
F	3	A
G	~~6~~ 5	~~E~~ D

Priority Queue
(C, 6)



Step 7: Extract C – Final Distances from A

Node	Dist(A, Node)	Prev
A	0	–
B	2	A
C	~~9~~ 6	~~B~~ E
D	~~5~~ 4	~~A~~ E
E	3	B
F	3	A
G	~~6~~ 5	~~E~~ D

Priority Queue
empty

